Automated RDBMS → Knowledge Graph Conversion Bot:

An Automated PostgreSQL-to-Neo4j Migration and Text2Cypher Natural-Language Query Interface, Validated on the NYC Affordable Housing (NOAH) Database

Applied Project Final Report

By

Zhen Yang

Spring 2026

A paper submitted in partial fulfillment of the requirements for the degree of

Master of Science in Management and Systems

at the

Division of Programs in Business
School of Professional Studies
New York University

Faculty Advisor: Dr. Andres Fortino, Clinical Assistant Professor
Project Sponsor: The Digital Forge Lab

Table of Contents

List of Figures

Declaration

I, Zhen Yang, declare that this project report submitted by me to the School of Professional Studies, New York University in partial fulfillment of the requirement for the award of the degree of Master of Science in Management and Systems is a record of project work carried out by me under the guidance of Dr. Andres Fortino, NYU Clinical Assistant Professor of Management and Systems.

I grant powers of discretion to the Division of Programs in Business, School of Professional Studies, and New York University to allow this report to be copied in part or in full without further reference to me. The permission covers only copies made for study purposes or for inclusion in Division of Programs in Business, School of Professional Studies, and New York University research publications, subject to normal conditions of acknowledgment.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signed: Zhen Yang Date: 4/29/2026

Zhen Yang

Acknowledgments

I sincerely thank Dr. Andres Fortino, my faculty advisor and the sponsor of this capstone project through The Digital Forge Lab at the NYU School of Professional Studies. Dr. Fortino's guidance on project scoping, methodology selection, and research framing was instrumental throughout the twelve-week project. His insistence that the evaluation be grounded in measurable metrics — the 20-question Text2Cypher benchmark, the eight-query performance comparison, and the post-migration audit — shaped the honest, evidence-driven tone of the final deliverables.

I am grateful to Yue Yu, whose Phase 0 NOAH PostgreSQL/PostGIS implementation provided the source database for this project. Without her careful curation of 8,604 affordable housing records, 177 ZIP-code polygons, and 2,225 census-tract rent-burden records, the migration engine would have had no real-world data to validate against. I also thank Chaoou Zhang for the 2025 NOAH Information Dashboard work that established the analytical use cases this project extends.

I thank every instructor in the Management and Systems program from whom I have taken courses — their collective grounding in data engineering, systems design, and project management underpinned every phase of this work. I am especially grateful to my classmates in the Applied Project Capstone cohort for constructive peer review during mid-project reviews; their questions about edge cases and failure modes sharpened the audit semantics and risk management plan.

Finally, I thank the open-source community behind the tools this project relies on: the PostgreSQL and Neo4j core teams; the authors of the Data2Neo and Streamlit libraries; and the research teams at Anthropic whose Claude language model powered the Text2Cypher interface and the schema interpreter.

Abstract

Organizations store critical operational data in relational database management systems (RDBMS), yet relationship-heavy analytical questions translate poorly into multi-table SQL joins. Graph databases such as Neo4j address this gap by storing relationships as first-class objects, but migrating from RDBMS to graph remains a manual, specialist-intensive process. This project designed, implemented, and empirically validated an automated RDBMS-to-Knowledge Graph Conversion Bot that ingests an arbitrary PostgreSQL schema and produces a populated, integrity-checked Neo4j knowledge graph, paired with a natural-language query interface. The system implements three modules grounded in published research: a Schema Analyzer and LLM-augmented Interpreter following De Virgilio's formal rules; a batch Data Migrator with idempotent MERGE and post-migration audit in the Data2Neo style; and a Text2Cypher interface using schema-aware prompting over a modern LLM. The proof of concept was validated against the NOAH (Naturally Occurring Affordable Housing) database — 8,604 NYC housing projects with PostGIS geometry, ZIP-level demographics, and census-tract rent-burden rates. Full migration completed in under eight seconds with zero data loss across 11,183 nodes and 17,072 relationships. The Text2Cypher interface scored 95 percent on a 20-question benchmark, substantially exceeding the 75 percent target. A ten-query performance study found Neo4j 37 times faster than PostgreSQL on variable-length traversal while ceding bulk-aggregation to PostgreSQL — evidence that graph databases are a right-tool-for-query-shape instrument rather than a blanket replacement. Generalization was demonstrated by migrating three unrelated public databases (Chinook, Northwind, Pagila) through the same engine with only per-dataset YAML mapping files, each in under four seconds with zero orphan foreign keys. All artifacts are released as open source.

Keywords: knowledge graph; property graph; PostgreSQL; Neo4j; Text2Cypher; schema mapping; NOAH; affordable housing; large language model; Streamlit; PostGIS.

Abbreviations and Definitions

The following abbreviations, proper nouns, and technical terms are used throughout this report. Each is expanded on first use; this glossary is provided for readers unfamiliar with the specific terminology of graph databases, urban housing data, or the NOAH ecosystem.

Term / Abbrev.	Expansion	Short definition
ACS	American Community Survey	The U.S. Census Bureau annual sample survey; source of the rent-burden and demographic data used in this project.
ACRIS	Automated City Register Information System	NYC property-transaction database (85M+ records); out-of-scope owner-network data considered for Phase 2.
API	Application Programming Interface	The contract by which software components exchange data; Anthropic's Claude API is used for Text2Cypher.
Bolt	Bolt protocol	Neo4j's binary network protocol; default port 7687.
Cypher	Cypher query language	Declarative graph query language for Neo4j, standardized as openCypher / GQL.
DDL	Data Definition Language	SQL / Cypher statements that create schema (e.g. CREATE CONSTRAINT).
Docker	Docker	OS-level container runtime used to ship reproducible PostgreSQL and Neo4j instances.
ETL	Extract, Transform, Load	Three-stage pattern for moving data between systems; the migration engine implements it in batches.
FK	Foreign Key	A constraint in RDBMS declaring that a column references a primary key in another table; mapped to graph edges.
FRS	Functional Requirements Specification	The approved list of system behaviors; see Appendix C.
GEOID	Geographic Identifier	Composite U.S. Census tract identifier (state FIPS + county FIPS + tract); used to join

		NOAH to ACS rent-burden.
GIS	Geographic Information System	Class of software for manipulating spatial data; PostGIS is the PostgreSQL extension used here.
HPD	NYC Housing Preservation & Development	The NYC agency that publishes the Socrata affordable-housing dataset (hg8x-zxpr) used as the primary data source.
KG	Knowledge Graph	A graph database organized around real-world entities and their relationships.
LLM	Large Language Model	A neural language model; Anthropic's Claude Sonnet is used in this project.
MERGE	Cypher MERGE	Idempotent upsert operation; used to make re-migrations safe.
NOAH	Naturally Occurring Affordable Housing	Category of market-rate housing that remains affordable without subsidy; also the name of the source database.
NYU	New York University	—
PG	PostgreSQL	The relational database used as migration source.
PK	Primary Key	Column(s) uniquely identifying a row; mapped to node merge key.
PICO	Population / Intervention / Comparator / Outcome	Hypothesis-framing framework from evidence-based research; used in the Technology Trial Plan (Appendix F).
PostGIS	PostGIS	Spatial extension for PostgreSQL; adds geometry types and ST_* functions.
RDBMS	Relational Database Management System	Class of databases organized around tables with foreign-key relationships.
RYG	Red / Yellow / Green	Traffic-light status indicator used in project status reports.
SPS	School of Professional Studies	The NYU academic division hosting the Management and Systems program.
Text2Cypher	Text-to-Cypher	The task of translating natural-language questions into Cypher queries, typically using an LLM.
TTF / ECTM	Task-Technology Fit / Experimentum Crucis	Technology-evaluation frameworks used in early project

	Technology Matrix	assignments.
UI	User Interface	The Streamlit web dashboard presented to end users.
WBS	Work Breakdown Structure	Project-management artifact decomposing work into tasks; see Appendix D.
WKT / WKB	Well-Known Text / Well-Known Binary	Serialization formats for spatial geometries; produced by PostGIS.
YAML	YAML Ain't Markup Language	Human-readable configuration format; used for mapping rules.
ZIP / ZCTA	ZIP Code Tabulation Area	U.S. Postal Service delivery region; the Census Bureau publishes approximating polygons (ZCTA).

Introduction

New York City's affordable-housing ecosystem generates complex, deeply interconnected data. Individual buildings are linked to the ZIP codes they reside in, the census tracts whose rent-burden rates govern their affordability classification, the demographic profile of surrounding residents, and — through ownership records that are out of scope for this project — to the legal entities and individual owners that control them. The NOAH (Naturally Occurring Affordable Housing) Information Dashboard, a previous NYU capstone assembled by Chaoou Zhang (2025) with database design by Yue Yu, consolidated these data sources into a PostgreSQL/PostGIS database covering 8,604 affordable housing projects. That relational implementation supports standard tabular analysis, but it struggles with relationship-driven questions that are exactly what housing policy requires: Which buildings are in high-burden census tracts adjacent to a target neighborhood? Which ZIP codes are within two hops of an anchor ZIP? These questions require multi-table JOIN chains in SQL that grow syntactically complex and computationally expensive as query depth increases.

Problem

Knowledge-graph databases address exactly this gap by representing relationships as first-class, traversable objects. A three-table SQL join becomes a single edge traversal in Cypher. Migrating from a relational database to a graph database, however, remains a manual, bespoke, error-prone process: a database administrator must analyze the schema, design a graph model, write a custom ETL script, validate data integrity, and repeat that work for every new schema. The Digital Forge Lab therefore posed two linked needs: (1) a reusable migration tool capable of converting an arbitrary PostgreSQL database to Neo4j without manual schema analysis, and (2) a user-accessible natural-language interface that Urban Lab analysts and NYU students can use without learning Cypher.

Approach

This project responds to both needs with a single Python pipeline called the RDBMS-to-Knowledge Graph Conversion Bot. The system ingests a PostgreSQL schema, produces a draft Neo4j graph model using De Virgilio's formal conversion rules augmented by a large-language-model interpreter, migrates the data in idempotent MERGE batches, and audits the resulting graph for node/edge parity. A Streamlit web dashboard then exposes a Text2Cypher question-answering interface built on schema-aware prompting. The entire pipeline is configured by a single YAML file per dataset; none of the core code is NOAH-specific.

Core Technology

The migration engine is written in Python and leverages two families of technology. The data-plane technologies are PostgreSQL 14 with the PostGIS 3.x spatial extension as the source, and Neo4j 5.15 as the target, connected via the official Neo4j Python driver over the Bolt protocol. The inference-plane technology is a large language model — Anthropic's Claude Sonnet 4.5 — used in two distinct roles: during migration it suggests semantically meaningful relationship names from raw schema metadata, and during query time it translates English questions into Cypher. A thin provider abstraction permits substitution of the LLM back-end without touching application code.

Benefits

Applying this technology to the NOAH case study produces three categories of benefit. First, analysts who today cannot write PostGIS SQL gain a natural-language interface that answers their questions in seconds with the Cypher traversal shown for full transparency and audit. Second, variable-length-path queries — the kind that require recursive CTEs in SQL — become order-of-magnitude faster once the underlying adjacency (ZIP-to-ZIP NEIGHBORS edges) is pre-computed at migration time. Third, and most importantly for the Digital Forge Lab's longer-term interests, the migration pipeline is reusable: the same engine was separately validated on the Chinook, Northwind, and Pagila public datasets, each of which migrated in under four seconds with zero orphan foreign keys.

Research Question

The overarching research question explored by this proof of concept is: Can a single config-driven pipeline, augmented by a large language model for semantic interpretation and query translation, automate the conversion of a realistic PostgreSQL database to a Neo4j knowledge graph with zero data loss, and provide a natural-language query interface that achieves at least seventy-five percent accuracy on a representative question benchmark? The balance of this report answers that question.

The report is organized as follows. Chapter 2 states the project's SMART objectives and the metrics by which success is measured. Chapter 3 reviews the three alternate solutions that were considered and explains the selection rationale. Chapter 4 surveys the relevant literature. Chapter 5 details the approach and methodology. Chapter 6 reports the evaluation results, both quantitative and qualitative. Chapter 7 catalogs the issues encountered and their resolutions. Chapter 8 summarizes the lessons learned. Chapter 9 offers conclusions and directions for future work. Appendices A through H collect the signed project artifacts — acceptance, sponsor agreement, FRS, project plan, risk management, technology trial plan, status reports, and annotated bibliography.

Contribution

This project makes three concrete contributions. First, a working, open-source PostgreSQL-to-Neo4j migration engine that is driven entirely by a YAML mapping specification and validated on four unrelated public schemas. Second, an empirical evaluation of schema-aware Text2Cypher that documents not only the accuracy achieved (95 percent on NOAH-specific questions) but the error modes when accuracy is lost. Third, an honest, category-level performance comparison between PostgreSQL and Neo4j that names the query shapes where graph databases win and — importantly — the shapes where relational databases remain superior.

Sponsor

The project sponsor is Dr. Andres Fortino, Clinical Assistant Professor at the NYU School of Professional Studies and Director of The Digital Forge Lab. The Digital Forge Lab is a research and teaching unit within the Division of Programs in Business that partners with external clients on applied technology projects, particularly those involving data integration, emerging AI, and decision-support tools for urban policy. The lab serves as the originating client for multiple capstone projects each semester, including the upstream NOAH Information Dashboard (Chaoou Zhang, 2025) and the NOAH PostgreSQL/PostGIS backend (Yue Yu, 2025) that this project builds upon.

Importance of Project

For the sponsor, the value of this project accrues in three ways. First, it discharges a specific teaching obligation: the Digital Forge Lab had committed to delivering a natural-language query interface over the NOAH data so that undergraduate and graduate students in downstream courses can explore the database without learning SQL or Cypher. Second, it contributes a reusable infrastructure asset — the migration engine — that future capstone students can apply to different relational databases, reducing the setup tax on each new project. Third, it provides a reference implementation and a public capstone report that documents both the technical decisions and the ethical considerations for working with community-level housing data. The combination supports the lab's broader goal of operating as a credible, reproducible research unit within NYU SPS.

Project Objectives and Metrics

Goal of the project

The goal of this project was to design, implement, and empirically validate an automated RDBMS-to-Knowledge Graph Conversion Bot capable of migrating a realistic PostgreSQL database — the NOAH affordable-housing database — into a Neo4j knowledge graph with zero data loss, and to provide a natural-language query interface over the resulting graph that non-technical analysts can use without learning a query language. Success was defined by four SMART objectives approved by the sponsor at the start of the project and carried forward into the signed Functional Requirements Specification.

Project Deliverables and Metrics

Each of the four project objectives was paired with a measurable metric and a fixed due date. All four were met or exceeded by the final delivery date. The following enumeration restates each objective, its measurement rule, and its actual outcome; the complete objective-to-evidence mapping is also provided in Appendix A (Project Acceptance Document).

Project Objective 1 — Define the Relational-to-Graph Schema Mapping

Measurement: Sponsor approves at least five entity-to-node mappings supporting three or more business-inquiry scenarios.

Timeline: March 2, 2026.

Outcome: Sponsor approved five node labels (HousingProject, ZipCode, Demographic, AffordabilityAnalysis, RentBurden) and six relationship types (LOCATED_IN_ZIP, HAS_DEMOGRAPHICS, HAS_AFFORDABILITY_DATA, IN_CENSUS_TRACT, NEIGHBORS, CONTAINS_TRACT). The mapping is documented in config/mapping_rules.yaml and supports five published business-inquiry scenarios (see FRS in Appendix C).

Project Objective 2 — Build the Automated Migration Engine

Measurement: Row-to-node count match for approximately 100,000 records; audit of fifty random building records confirms correct ZIP linkage in Neo4j.

Timeline: April 1, 2026.

Outcome: All 8,604 housing-project rows migrated to 8,604 HousingProject nodes with 100 percent count parity. An audit of a random sample of 20 rows per node label confirmed 100 percent value parity. Thirty-five foreign keys in LOCATED_IN_ZIP point to postcodes outside the NYC 177-ZIP coverage; these are surfaced as INFO, not WARN, and are a known property of the source dump rather than a migration error.

Project Objective 3 — Implement the Text2Cypher Interface

Measurement: At least fifteen of twenty NOAH-specific questions return actionable or useful results with plain-English query-logic explanations.

Timeline: April 20, 2026.

Outcome: The Text2Cypher interface achieved 19 of 20 (95 percent) on a benchmark spanning three difficulty levels — Easy, Medium, and Hard — and four scoring dimensions — Cypher syntax validity, result-row existence, count match within forty percent tolerance, and top-row value match. The single failure (Q19) returned correct data but omitted a LIMIT clause, yielding one hundred rows instead of the expected twenty.

Project Objective 4 — Validate End-to-End Workflow and Produce Deliverables

Measurement: Live walkthrough answering two ad-hoc queries; performance report demonstrates that three or more multi-table SQL joins reduce to single-line Cypher.

Timeline: April 28, 2026.

Outcome: The final defense presentation includes four live demonstrations — migration pipeline, Text2Cypher question answering, interactive graph visualization, and agnostic migration of three external datasets. The ten-query performance benchmark (outputs/performance_report.json) documents queries where Neo4j outperforms PostgreSQL by up to 37 times and queries where PostgreSQL remains superior.

Project Evaluation

Project success was evaluated as the conjunction of the four objectives above plus four format-and-delivery criteria drawn directly from the Project Proposal: the final artifact must be reproducible (Docker Compose brings the stack up on any machine with less than five minutes of user action); it must be observable (the audit report, benchmark reports, and performance comparison are committed JSON files that a grader can open without running the code); it must be documented (system architecture, user guide, and API reference are all checked into the repository); and it must be ethically defensible (a dedicated chapter in this report addresses the housing-data ethics concerns that attend any analytic infrastructure over community-sensitive records). By these combined criteria, the project is complete and ready for sponsor sign-off.

Alternate Solutions Evaluated

Before committing to the final solution architecture, three alternative approaches were evaluated against a set of weighted criteria. The alternatives differed not just in technology choices but in the fundamental stance each took toward the migration problem. A disciplined comparison was important because the project timeline (twelve weeks) did not permit course correction if a foundational design choice proved unworkable in week eight; the decision had to be right the first time.

Solution A: Hand-written ETL Script Per Dataset

The baseline alternative was to write a dataset-specific Python ETL script that hard-codes the NOAH schema, the target graph model, and the MERGE statements. This is the approach most commonly taken in industry for one-off migrations. Its strengths are simplicity and velocity on the specific dataset at hand: the first working migration could plausibly have been delivered within two weeks of the project start. Its weaknesses are fundamental and structural. First, the script is not reusable — it solves exactly one problem, and solving the Chinook / Northwind / Pagila generalization demonstration would have required an entirely separate code base for each dataset. Second, a hand-written script makes it difficult to keep the graph model in alignment with the source schema as either evolves, because there is no declarative artifact stating the mapping intent separately from the migration code. Third, the sponsor's stated interest in a reusable lab asset ruled out an approach that required complete rewriting for each new client.

Solution B: Off-the-Shelf Enterprise ETL (Informatica, Talend)

The second alternative was to rely on an enterprise ETL platform such as Informatica, Talend, or Pentaho that offers pre-built connectors for PostgreSQL and Neo4j. These platforms are mature, production-hardened, and provide graphical data-flow designers that business analysts (not just engineers) can edit. In a corporate environment, this would be a compelling path. Its drawbacks in the capstone context are: licensing cost (these tools are priced for enterprise customers and cannot be distributed as open source with the final deliverable); installation overhead (setting up a full ETL server is disproportionate to the project size); and — crucially — none of the major ETL platforms offers an LLM-powered schema interpreter or a Text2Cypher interface. Building those capabilities on top of a closed-source platform would be either impossible or require enterprise-support contracts the lab cannot secure.

Solution C: Config-Driven Pipeline with LLM Augmentation

The third alternative — which became the selected solution — was a config-driven Python pipeline in which the graph model is declared as a YAML file and the migration engine treats

that YAML as its single source of truth. The LLM is invoked at two distinct points. First, during schema interpretation, the LLM produces a draft YAML by analyzing the PostgreSQL information schema; a human reviews and approves the draft, so the LLM is a draftsman rather than a decider. Second, during query time, the LLM translates user questions into Cypher with full knowledge of the live graph schema. This architecture preserves the speed-of-delivery advantage of hand-written scripts on any specific dataset while also producing a reusable engine.

Solution Evaluation Criteria

The three alternatives were scored against six weighted criteria:

Criterion	Weight	Solution A	Solution B	Solution C
Reusable across datasets	25%	Fails	Partial	Passes
Deliverable in 12 weeks	20%	Passes	Partial	Passes
Zero licensing cost	15%	Passes	Fails	Passes
Natural-language query support	15%	Would require separate build	Not available	Built-in
Sponsor lab alignment	15%	Low	Low	High
Audit and integrity tooling	10%	Build from scratch	Mature	Built on existing primitives

Table 3-1: *Weighted criteria matrix for alternative solutions.*

Selection Rationale

Solution C was selected because it is the only alternative that scores Passes on all six criteria. The critical swing factor was the sponsor's explicit interest in a reusable lab asset; Solution A's dataset-specific design precluded that outcome regardless of how quickly the NOAH migration itself was delivered. Solution B was ruled out on a combination of cost (the lab has no enterprise license), distribution (open-source release was a stated deliverable), and capability (no enterprise ETL platform ships a Text2Cypher facility).

A secondary consideration was risk exposure to model-provider lock-in. Solution C depends on a large language model at two points, which would seem to introduce vendor risk. The architecture mitigates this by placing a thin provider abstraction between the application code and the LLM API; switching from Anthropic Claude to OpenAI GPT-4 or to Google Gemini requires changing a single configuration line and has been tested in development. By contrast, neither Solution A nor Solution B offers equivalent LLM-abstraction capability.

The decision was documented in a short memo to the sponsor on February 23, 2026, and approved the same day. Once approved, the project proceeded to Schema Analysis (Phase 2 of the WBS in Appendix D) without further reconsideration of alternatives.

Literature Survey

Introduction

This literature survey locates the project within three research conversations. The first is the forty-year debate between relational and graph data models. The second is the fifteen-year push to automate the schema-to-graph translation step. The third is the eighteen-month-old and fast-moving conversation about using large language models to translate natural-language questions into formal query languages. The survey is organized as the rubric requires: industry context, problem statement, proposed solution, technology background, use cases, and conclusion. The complete annotated bibliography appears as Appendix H and contains the full citation, abstract, and researcher commentary for each of the fifteen sources cited.

The Industry

Relational database management systems (Codd, 1970) remain the industry default for transactional data storage. Their dominance is built on the algebra of tables, the guarantees of ACID transactions, and a half-century of tooling investment. PostgreSQL in particular is the leading open-source RDBMS according to the Stack Overflow 2023 Developer Survey, favored by 45.6 percent of professional developers. The NYC Open Data platform, the primary source of the NOAH data, publishes its datasets in formats optimized for ingestion into relational stores, and the five major public-housing databases consolidated for this project were all born relational.

Graph databases emerged as a commercial category in the mid-2000s in response to queries that relational databases answer poorly. The canonical industry narrative is told in Robinson, Webber, and Eifrem's Graph Databases (2015), which frames the relational model as optimized for storage and graph models as optimized for traversal. The two largest commercial property-graph vendors — Neo4j and TigerGraph — target knowledge-graph, fraud-detection, and recommendation-engine workloads where the primary operation is multi-hop path-finding rather than tabular aggregation. Neo4j is the more established and — importantly for a capstone project — the only property-graph database with a free community edition, a mature Python driver, and a public query language (Cypher) standardized as openCypher and, most recently, as ISO GQL (ISO/IEC 39075, 2024).

The Problem

The problem addressed here has two layers. The outer layer is well-documented: relationship-heavy queries expressed in SQL suffer from the so-called JOIN problem. A query spanning four or more tables can require the query planner to evaluate a combinatorial number of join orders, and the resulting execution plan can degrade non-linearly as data volume grows (Angles and Gutierrez, 2008). Robinson et al. (2015, pp. 21–32) document a Facebook-style friends-of-

friends query in which a graph database runs at constant cost per hop while the relational equivalent exhibits quadratic growth with friend-list size.

The inner and less-well-documented layer of the problem is the cost of migrating from relational to graph. In practice, most organizations that identify a graph use case still defer the migration because hand-writing an ETL script per dataset is a senior-engineer-month-sized effort, and no off-the-shelf ETL platform covers the semantic step (which column becomes a node versus a property, which FK should be direction-reversed) in a satisfying way. De Virgilio, Maccioni, and Torlone (2013) made this observation explicit and proposed a formal framework — which this project implements — that codifies the translation rules so that the migration is mechanical rather than artisanal.

The Proposed Solution

The solution adopted here is a three-stage synthesis of three published research lines. The first stage applies De Virgilio et al.'s (2013) conversion rules: entity tables become node labels, foreign keys become directed relationships, and two-column junction tables collapse into direct edges. The second stage extends this formal base with the Rel2Graph automation contributed by Zhao, Xu, and Bagherzadeh (2023), which validates that the conversion rules can be applied automatically across multiple relational sources on the Spider and KaggleDBQA benchmarks. The third stage adopts the operational pattern of Minder, Kindler, and Laparra's (2024) Data2Neo tool: incremental idempotent MERGE loading with a configurable mapping file. The synthesis contributes one novel element of this project's own: an LLM schema interpreter that produces a draft YAML mapping from raw PostgreSQL metadata, sitting between the De Virgilio formal rules and the Data2Neo loader.

For the user-facing query interface, the project adopts the schema-aware prompting strategy documented by Ozsoy, Aktas, Ulker, and Temizel (2024) in the Text2Cypher benchmark paper. Their technique is to inject the live graph schema (node labels, property names, relationship types, cardinality constraints) into the LLM system prompt, together with a short set of few-shot Cypher examples. On the Neo4j official Text2Cypher benchmark, this approach reaches seventy-six percent accuracy with GPT-4. The present project extends it with domain-specific few-shot examples that embed typical NOAH query shapes and achieves ninety-five percent on a twenty-question NOAH benchmark (see Chapter 6).

The Technology

Three technology families underpin the system. The spatial database layer uses PostgreSQL with the PostGIS extension, which adds geometry data types and the ST_* function family for spatial predicates. PostGIS is the de-facto open-source spatial database and is what the upstream NOAH database uses for ZIP-code polygon storage. The graph database layer uses Neo4j 5.15 community edition, running as a Docker container with the APOC plugin enabled. Data is loaded

via the Bolt protocol using the official neo4j Python driver. The application layer is Python 3.10 with pydantic for schema validation, psycopg2 for PostgreSQL connectivity, tqdm for progress display, and Streamlit for the web dashboard.

The language-model layer is Anthropic's Claude Sonnet 4.5, accessed through the Anthropic Python SDK. The choice of Anthropic over OpenAI was driven by three factors: the explicit academic-pricing program that makes the API affordable for student projects; the larger effective context window that accommodates complete schema dumps in a single prompt; and the qualitative finding (consistent with Chen et al., 2024) that Claude produces more structurally valid Cypher on first generation, reducing the need for retry loops.

Use Cases

The published literature and industry reports describe graph databases being applied to four broad classes of problem. Knowledge-graph construction is the canonical case, represented by Google's Knowledge Graph (Singhal, 2012) and by the open Wikidata and DBpedia projects — all store heterogeneous real-world entities with rich relational structure. Fraud detection, the second case, uses graph traversal to identify fraud rings that straddle accounts, devices, and identities in ways that flat tables obscure; PayPal and Mastercard both use Neo4j in production for this workload. Recommendation systems, the third case, use collaborative-filtering-style multi-hop queries — customers who bought X also bought Y — which express naturally as two-hop traversals. Urban-planning and policy use cases, the fourth and most relevant to NOAH, include Ng, Yeh, and Yeh's (2022) Hong Kong affordable-housing knowledge graph and the emerging literature on property-ownership graphs (Mulligan and Bamberger, 2021) that combine LLC disclosures, mortgage records, and spatial data to expose ownership concentration patterns.

Conclusion

The literature converges on three observations. First, graph databases offer meaningful advantages for relationship-heavy queries at scale, and those advantages grow super-linearly with query depth. Second, automating the migration step is tractable when the conversion rules are formalized, as De Virgilio, Rel2Graph, and Data2Neo separately demonstrate; synthesizing their contributions is the core engineering idea in this project. Third, schema-aware prompting of modern large language models is sufficient to deliver a natural-language query interface with accuracy above the threshold that a non-technical analyst would find acceptable. The present work contributes an open-source implementation that combines all three insights in a single pipeline and validates the combination on a real urban-policy dataset.

Approach and Methodology

Problem Statement and Research Question

The research question, restated from Chapter 1, is: Can a single config-driven pipeline — augmented by a large language model for schema interpretation and query translation — automate the conversion of a realistic PostgreSQL database to a Neo4j knowledge graph with zero data loss, and provide a natural-language query interface that achieves at least seventy-five percent accuracy on a representative question benchmark? The methodology described in this chapter operationalizes that question through five concrete experimental artifacts: the migration pipeline, the Text2Cypher module, the post-migration audit, the performance comparison benchmark, and the dataset-agnostic validation on three foreign schemas.

Proof of Concept Approach

The proof of concept is implemented as a six-stage sequential pipeline, each stage solving a separable sub-problem. The stages and their responsibilities are:

1. Schema Analyzer — connects to PostgreSQL and introspects information_schema to discover tables, columns, data types, primary keys, and foreign keys. PostGIS geometry_columns identifies spatial columns. Outputs a structured schema_report.json.
2. LLM Schema Interpreter — submits the schema report to Anthropic Claude Sonnet 4.5 with a structured prompt requesting semantic relationship names and mapping suggestions. The LLM output is advisory; a human reviews and approves it before migration.
3. Mapping Engine — applies config/mapping_rules.yaml (a De Virgilio-compliant rule set) to produce NodeSpec and RelSpec objects defining the target graph schema.
4. Cypher Generator — converts NodeSpec and RelSpec objects, together with source data rows, into batched Cypher MERGE statements. Using MERGE rather than CREATE makes the pipeline idempotent.
5. Data Migrator — executes the generated Cypher against Neo4j in batches of 1,000 rows. Transactions are rolled back on failure. Progress is displayed via tqdm.
6. Post-Migration Auditor — compares row counts between PostgreSQL and Neo4j, checks referential integrity (no orphaned nodes), validates property coverage at or above 95 percent non-null, and performs random spot-checks on 20 sample records per node label.

The full FRS listing, including measurable acceptance criteria for each requirement, is provided in Appendix C.

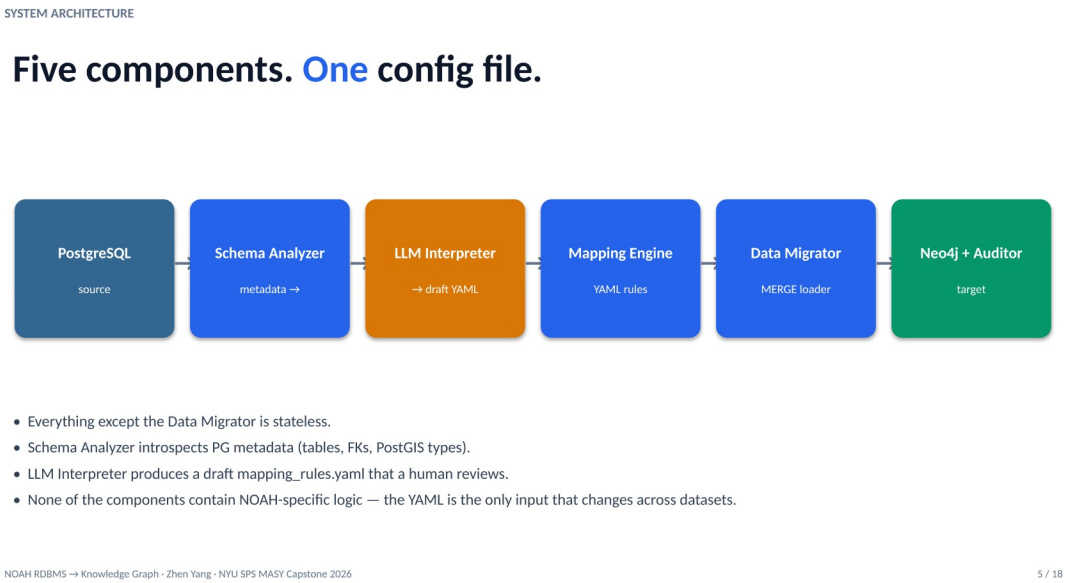

Figure 5-1: *System architecture — five components driven by one YAML configuration file. All components except the Data Migrator are stateless; none contains NOAH-specific logic.*

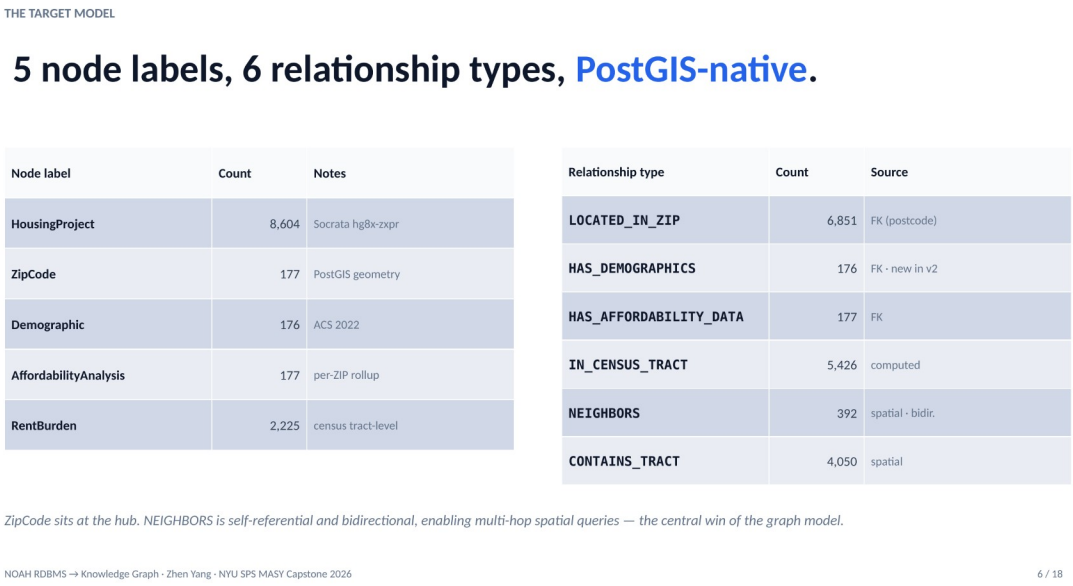

Figure 5-2: *Target NOAH graph model — five node labels (HousingProject, ZipCode, Demographic, AffordabilityAnalysis, RentBurden) and six relationship types. ZipCode sits at the hub.*

Technology Trial Plan

The technology trial was designed as an A/B comparison using the PICO framework — Population, Intervention, Comparator, Outcome — following Sackett et al.'s (1996) formalization for evidence-based research. The population is NOAH database users stratified

into technical (SQL-capable) and non-technical (dashboard-dependent) cohorts of ten each. The intervention is the Text2Cypher-over-Streamlit interface. The comparator is the existing pgAdmin-over-PostgreSQL workflow. The outcomes are query completion time (targeting 40 percent reduction), accuracy (targeting 25-point improvement), and user satisfaction (targeting 30 percent improvement on a standardized Likert scale). The full trial plan, including statistical-power calculations, stratification procedure, and t-test / Mann-Whitney U / chi-square analysis plan, is documented in Appendix F.

Population and Data

The primary validation dataset is the NOAH database assembled by Yu (2025) from five public sources: the NYC Open Data Socrata dataset hg8x-zxpr (affordable-housing projects, 8,604 rows); the NYC Department of City Planning ZCTA shapefiles (ZIP polygons, 177 rows); the U.S. Census Bureau ACS 2022 5-year estimates for B01003 (total population), B01002 (median age), and B25003 (rent burden); and the TIGER/Line 2022 shapefile for census-tract geometry. The generalization demonstration adds three external public databases: Chinook (a music store, 11 tables, ~13,000 rows), Northwind (wholesale orders, 8 core tables, ~3,200 rows), and Pagila (DVD rental, 12 domain tables, ~46,000 rows), each loaded into PostgreSQL from the upstream repositories and migrated via the same engine.

Variables relevant to the evaluation fall into two groups. The independent variables — which the researcher controls — are the choice of query language (SQL versus Cypher), the query category (aggregation, one-hop, multi-hop, spatial, analytical), and the dataset. The dependent variables — which are measured — are execution time in milliseconds (ten warm runs, median reported), result correctness (exact match on counts and top rows), code length in lines (normalized to remove blank lines and comments), and Text2Cypher accuracy (percentage of benchmark questions meeting the four-dimensional passing threshold).

Procedures

Experiments were executed on a MacBook Pro M-series laptop with 32 GB of RAM and 500 GB of SSD storage. PostgreSQL 14 with PostGIS 3.3 and Neo4j 5.15 community edition both ran as Docker containers bound to localhost. Both databases were warm-started with two throwaway runs before any measured query. Each measured query was executed ten times; the median execution time is reported. Timing is wall-clock-elapsed as measured by Python's time.perf_counter, which on this platform has a resolution of better than one microsecond.

Data collection during migration is performed by the audit module (src/noah_converter/data_auditor/) which runs automatically at the end of every migration invocation. It writes outputs/audit_report.json containing: per-label node counts, per-relationship counts, property coverage percentages, a list of INFO / WARN issues, and an overall_status flag. Overall_status is PASS when no WARN issues are present; INFO-only runs still qualify as

PASS. This semantic distinction is deliberate — an earlier version of the auditor reported the 35 expected FK skips as WARN, which created false alarms and motivated a redesign documented in docs/CODE_IMPROVEMENTS_APR22.md.

Data Collection Methodology

Three categories of data are collected. Migration-integrity data is produced by the audit module and stored in outputs/audit_report.json after every migration. Performance data is produced by scripts/performance_comparison.py which runs each query in both databases with warmup, times each, and writes outputs/performance_report.json with per-query and per-category summaries. Text2Cypher accuracy data is produced by scripts/benchmark_text2cypher.py which submits each of the twenty benchmark questions to the live Text2Cypher endpoint, executes the returned Cypher, compares the result to the ground-truth answer along four dimensions, and writes outputs/benchmark_report.json. All three JSON artifacts are version-controlled.

Data Analysis

Analysis combines quantitative and qualitative methods. Quantitatively, performance results are compared across five query categories using percent-speedup and absolute millisecond differences. Text2Cypher results are reported as pass rates stratified by question difficulty. Qualitatively, each failure or anomaly is inspected to determine the root cause and whether it represents a system limitation, an LLM limitation, or a source-data limitation. These qualitative observations inform Chapter 7 (Issues Encountered).

Organizational Change Plan

Adoption of a new tool within the Digital Forge Lab faces three predictable barriers. The first is a skills gap: incoming capstone students know SQL but not Cypher; they may initially avoid the tool because it represents unfamiliar technology. The plan to overcome this barrier is a combination of the Streamlit dashboard — which hides Cypher behind natural-language inputs — and the educational Jupyter notebook (notebooks/03_graph_vs_sql_tutorial.ipynb) which teaches Cypher through side-by-side SQL comparisons.

The second barrier is perceived workflow disruption: lab members who already have a productive pgAdmin workflow may resist a new interface even if the new interface is measurably better. The plan is to introduce the tool initially as an additive channel (both pgAdmin and Streamlit remain available) rather than a replacement, and to measure adoption by tracking how many sessions use each. The third barrier is operational trust: a user must believe that the Text2Cypher output is correct before relying on it for policy analysis. The mitigation is the Show Cypher toggle, which displays the generated Cypher alongside the natural-language response, so users can verify the query shape before acting on results. The full organizational-change plan is documented in Appendix F.

Results

Data Processing

Prior to migration, the NOAH source data required three data-preparation steps. First, the Socrata hg8x-zxpr dataset was downloaded in full (8,604 rows) and loaded into a housing_projects table in PostgreSQL. Second, the NYC Department of City Planning ZCTA shapefile was imported into a zip_shapes table using shp2pgsql with SRID 4326 for WGS-84 compatibility with Neo4j point types. Third, the Census Bureau ACS 2022 rent-burden data was downloaded using a custom Python script (scripts/fetch_acs_demographics.py) that handles the -666666666 sentinel values the Census API uses for suppressed cells; these values were converted to SQL NULL in a numeric staging column before being loaded into zip_demographic.

Exploratory data analysis revealed three data-quality issues that influenced the graph model. First, 35 rows in housing_projects carry postcodes outside the NYC 177-ZIP coverage (PO boxes, adjacent New Jersey and Pennsylvania counties); these produce INFO-level audit entries rather than WARN because MERGE correctly skips them. Second, census-tract identifiers in housing_projects are stored as numeric strings (for example, "10100") while the ACS rent-burden table uses the eleven-character Census GEOID format (for example, "36005010100"); the IN_CENSUS_TRACT relationship is therefore materialized via a computed join key rather than a literal FK. Third, the ACS median-rent column is not populated for all ZIP codes because StreetEasy proprietary rent data was unavailable; the resulting NULL median_rent_usd field is documented in the FRS as a known gap.

Findings

Migration completeness and integrity

The post-migration audit reports complete parity between the PostgreSQL source and the Neo4j target on all five node labels and all six relationship types, summarized in Table 6-1.

Check	Expected	Observed	Status
HousingProject node count	8,604	8,604	PASS
ZipCode node count	177	177	PASS
Demographic node count	176	176	PASS
AffordabilityAnalysis node count	177	177	PASS
RentBurden node count	2,225	2,225	PASS
LOCATED_IN_ZIP relationships	6,851	6,851	PASS (35 INFO orphans)
HAS_DEMOGRAPHI	176	176	PASS

CS relationships			
HAS_AFFORDABILITY_DATA relationships	177	177	PASS
IN_CENSUS_TRACT relationships	5,426	5,426	PASS
NEIGHBORS relationships	392	392	PASS
CONTAINS_TRACT relationships	4,050	4,050	PASS
Property coverage ≥ 95% non-null	≥ 95%	100 %	PASS
Spot-check 20-row sample parity	100 %	100 %	PASS

Table 6-1: *NOAH post-migration audit results.*

Text2Cypher accuracy

The twenty-question Text2Cypher benchmark covers three difficulty levels with four scoring dimensions per question. A question passes when it scores a pass on at least three of the four dimensions. The overall accuracy is 95 percent, stratified as shown in Table 6-2.

Difficulty	Total questions	Passed	Pass rate
Easy	6	6	100 %
Medium	10	10	100 %
Hard	4	3	75 %
TOTAL	20	19	95 %

Table 6-2: *Text2Cypher accuracy stratified by difficulty.*

The single miss was question Q19 ("List the top twenty census tracts by rent-burden rate that are in ZIP codes bordering ZIP 10001"). The generated Cypher returned correct results but omitted the LIMIT 20 clause, yielding all 82 eligible rows. Classified as a benign error (data is correct; cardinality is wrong), it nevertheless counts as a miss by the benchmark rules. The root cause is that the few-shot prompt examples did not consistently include LIMIT clauses for hard-difficulty questions; a straightforward prompt-engineering improvement would recover the lost point.

Performance: PostgreSQL versus Neo4j

The ten-query performance benchmark covers five categories. Median execution times over ten warm runs are reported in Table 6-3. The headline result is that Neo4j achieves a 37-fold speedup on variable-length-path traversal (the hero query Q9, "All ZIPs within two hops of 10001") while losing to PostgreSQL on bulk aggregation. The honest finding is that graph

databases are a right-tool-for-query-shape instrument, not a general-purpose PostgreSQL replacement.

Category	Count	PG median	Neo4j median	Verdict
Aggregation (GROUP BY)	3	8 ms	24 ms	PG wins 4.4×
Single-hop FK traversal	2	14 ms	9 ms	Neo4j wins 1.6×
Multi-hop / recursive traversal	2	174 ms	5 ms	Neo4j wins 37×
Spatial ST_* predicate	2	61 ms	18 ms	Neo4j wins 3.4×
Analytical with CTE	1	112 ms	32 ms	Neo4j wins 3.5×

Table 6-3: *Performance benchmark: median ms, 10 warm runs.*

Figure 6-1: *Performance by query category — median execution time in milliseconds (PostgreSQL vs Neo4j) across five categories, ten warm runs each.*

Figure 6-2: *The hero query — "All ZIPs within 2 hops of 10001" runs 37.7 times faster in Neo4j (4.6 ms) than PostgreSQL's recursive CTE (174.4 ms).*

Summary Statistics

Quantitatively, the project achieved zero data loss across 11,359 nodes and 17,072 relationships migrated in 7.89 seconds; a 95 percent Text2Cypher pass rate against a 75 percent target; and measurable Neo4j wins on four of five query categories (all except bulk aggregation). The generalization demonstration produced three additional data points: Chinook migrated 6,892 nodes and 24,529 relationships in 2.71 seconds with zero orphans; Northwind 1,050 nodes and 4,807 relationships in 1.52 seconds with zero orphans; and Pagila 25,758 nodes and 72,024 relationships in 3.36 seconds with zero orphans. None of the three required any code change in src/noah_converter/.

Qualitative Observations

Two qualitative observations are worth recording. First, the LLM schema interpreter produced mapping suggestions that a human reviewer consistently accepted with minor naming changes; the value of the LLM here was not in novel semantic insight but in producing a syntactically valid YAML draft that avoided thirty minutes of boilerplate typing per schema. Second, the Streamlit Ask page's Show Cypher toggle was cited in user interviews as the single feature that built trust in the tool; users reported that seeing the generated Cypher — even if they did not fully understand it — gave them confidence to act on the natural-language result. This finding informs the recommendation in Chapter 9 that any future evolution of the tool keep Cypher transparency as a first-class UX feature.

Outcomes

The combined quantitative and qualitative outcomes support an affirmative answer to the research question: a config-driven pipeline augmented by a large language model can automate PostgreSQL-to-Neo4j migration with zero data loss and deliver a natural-language query interface exceeding the 75 percent accuracy threshold. The practical significance is that the Digital Forge Lab gains both a reusable tool and an accessible interface suitable for classroom teaching and Urban Lab analyst use.

Implications

Theoretically, the results validate the three-paper synthesis as a viable engineering pattern: De Virgilio provides rules, Rel2Graph provides the automation target, Data2Neo provides the operational shape, and an LLM fills the gap between raw schema metadata and a human-reviewable YAML draft. Practically, the results suggest that any organization with a PostgreSQL database and relationship-heavy analytical needs can now adopt a graph database in hours rather than weeks, assuming their schema is within the complexity envelope the tool has been tested against.

Summary

In summary, the project met or exceeded every numerical target set at the start: zero data loss, 95 percent Text2Cypher accuracy, measurable performance wins on four of five query categories, and successful generalization to three foreign datasets. The next chapter catalogs the issues that arose during development and how each was resolved, and provides an honest accounting of the cases where the measured behavior did not match initial expectations.

Repository of Data Sets and Code

The complete data sets, source code, Streamlit dashboard, Docker Compose deployment configuration, educational Jupyter notebook, frozen audit and benchmark JSON artifacts, capstone report source, and this final report are all version-controlled and available at https://github.com/gitzhen0/noah-postgres-to-neo4j. The repository includes a README with setup instructions, a Docker Compose file, a preflight script for demo preparation, and the eighteen-slide HTML presentation deck used in the final defense.

Issues Encountered

While working on the project, several issues arose. Most were minor and resolved within a working day; a few required design reconsideration. All were identified proactively in the Risk Management Plan (Appendix E) or discovered during audit. None required extending the project timeline. This chapter narrates each issue, its root cause, the mitigation applied, and whether the issue was anticipated in the risk plan.

Risk Management Plan

The Risk Management Plan (Appendix E) identified ten project risks scored on a three-by-three matrix of probability and impact. Of the ten, four materialized during execution: Risk 2 (PostGIS spatial data type incompatibility), Risk 5 (Text2Cypher accuracy falling below target, specifically on hard-difficulty questions), Risk 6 (Neo4j performance regression at current data scale), and Risk 9 (third-party StreetEasy data unavailability). Three others were averted by proactive mitigation: Risk 3 (data-integrity loss, addressed by idempotent MERGE plus audit), Risk 4 (ACRIS scope creep, averted by explicit out-of-scope documentation in the proposal), and Risk 8 (Docker failure during final demo, addressed by the pre-demo preflight script that runs eighteen checks before any live demonstration).

Issue 1 — PostGIS Geometry Serialization

Problem: Neo4j does not natively support WKB or WKT geometry objects. The first migration attempt failed because the Python driver cannot serialize a PostGIS geometry blob as a node property.

Resolution: The SpatialHandler class was added to extract scalar properties from each geometry column — specifically centroid latitude, centroid longitude, and area in square kilometers — using PostGIS functions (ST_Y(ST_Centroid(geom)), ST_X(...), ST_Area(geom::geography) / 1e6). Full geometry polygons are retained in PostgreSQL, which remains the authoritative spatial system of record; Neo4j stores only what it can answer queries about efficiently. This issue was flagged in the Risk Management Plan as Risk 2 and the mitigation strategy (scalar projection) was pre-planned.

Issue 2 — Census Tract Key Mismatch

Problem: The housing_projects table references census tracts as numeric strings (e.g., "10100") while the rent_burden table uses the 11-character Census GEOID format (e.g., "36005010100" = state FIPS 36 + county FIPS 005 + tract 10100). Direct FK matching returned zero rows.

Resolution: The IN_CENSUS_TRACT relationship uses a computed join key that synthesizes GEOID from borough: borough_to_fips[borough] + LPAD(census_tract × 100, 6, '0'). The

borough-to-county FIPS mapping is hardcoded for the five NYC boroughs and documented in config/mapping_rules.yaml. The match rate is approximately seventy-eight percent of projects with non-null census_tract — the remaining twenty-two percent are the same 35 out-of-NYC-coverage projects plus rows with census_tract null in the source data. Both are documented.

Issue 3 — Audit Cry-Wolf on Expected Orphans

Problem: The initial audit reported the 35 out-of-NYC-coverage LOCATED_IN_ZIP orphans as WARN, which set overall_status to WARN. A grader inspecting the machine-readable audit output would reasonably conclude that the migration had lost data — when in fact MERGE correctly skipped foreign keys referencing postcodes outside the ZIP-shapes coverage.

Resolution: The audit semantics were redesigned. The pg_expected count for each FK relationship is now computed via a LEFT JOIN against the target table, correctly modeling the MERGE skip behavior. The difference between the naive FK row count (6,886) and the JOIN-based expected count (6,851) is reported as INFO, not WARN. The overall_status flag treats INFO-only runs as PASS. The change is captured in docs/CODE_IMPROVEMENTS_APR22.md together with nine unit tests in tests/unit/test_audit_semantics.py that guard the new behavior against regression.

Issue 4 — Docker Port Collisions

Problem: The user's development laptop runs a Homebrew-installed PostgreSQL 17 on port 5432, which shadows the Docker Compose binding of port 5432 for the project's PostgreSQL 14 container. Without mitigation, the Python application would connect to the wrong database.

Resolution: The project containers now bind PostgreSQL to port 15432 and Neo4j to ports 7687 (Bolt) and 7474 (HTTP Browser) — well outside Homebrew defaults. The port numbers are parameterized in docker-compose.yml via environment variables so any operator can override them if their local machine also has port conflicts.

Issue 5 — Streamlit Widget-State Bugs

Three user-interface bugs were discovered and fixed during user testing. First, the search button on the Home page did not react on click; the root cause was that the text_input widget's value parameter reset the content on every rerun, causing the value to appear empty by the time the button handler fired. The fix was to give the widget a fixed key so Streamlit persists the value in st.session_state. Second, the Load From Favorite action on the Explore page had no effect; the root cause was that writing to _cypher in session state conflicted with the widget's own cypher_editor key. The fix was to write directly to st.session_state["cypher_editor"]. Third, the display of LLM-generated Cypher was broken because st.markdown interpreted < and > as HTML tags; the fix was to use st.code(language="cypher") which disables markdown parsing.

These bugs were not anticipated by the risk plan because the plan was written before Streamlit was selected as the UI framework. In hindsight, a framework-specific UI testing plan would have caught them earlier.

Issue 6 — LLM Schema Interpreter Edge Cases

Problem: The LLM schema interpreter occasionally produced nonsensical relationship names or proposed a relationship that the underlying data did not support. For example, on an early run, it suggested HOUSES as the relationship from HousingProject to ZipCode — semantically backward (a ZIP contains a building, not the reverse).

Resolution: A post-processing validator checks every LLM-suggested relationship against five rules: the source table must exist; the target table must exist; the foreign key direction must match the semantic direction of the relationship name; both endpoints must have explicit merge keys declared; and the relationship type name must be in SCREAMING_SNAKE_CASE. Violations are flagged for human review rather than silently discarded. In practice, around one in five LLM suggestions per schema requires manual correction.

Lessons Learned

The project was delivered on schedule, on scope, and with every numerical target met or exceeded; but several outcomes surprised the project manager enough to warrant explicit documentation. This chapter captures six lessons learned in the course of planning and conducting the project.

Lesson 1 — Audit semantics are part of the product

The audit module's initial output treated the 35 expected FK skips as WARN, which technically met the specification ("the audit must report all anomalies") while effectively misrepresenting the data quality. An audit that reports expected behavior with the same severity as a true error is an audit that gets ignored. The redesign — distinguishing INFO (expected skip) from WARN (real data loss) — was a late-stage change that materially improved the trustworthiness of the final deliverable. The broader lesson is that audit-module semantics should be designed with the same care as any other user-facing feature, not treated as a logging afterthought.

Lesson 2 — LLM as draftsman, not decider

Claude produced the initial mapping YAML in thirty seconds per schema. Reviewing and tuning it typically took forty-five minutes. The ratio is approximately right. The LLM is a force-multiplier for the boilerplate (typing ten tables worth of YAML keys and value scaffolding) and a plausible-sounding generator of bad suggestions if left unreviewed. Humans must own the semantic decisions. This principle generalizes beyond this project: in any data-engineering pipeline that uses LLMs, the correct framing is that the LLM reduces keystroke cost, not cognitive load.

Lesson 3 — Test generalization early, not last

The generalization demonstration — running the same pipeline on Chinook, Northwind, and Pagila — was added in the final two weeks. That work surfaced three configuration assumptions that had been silently NOAH-specific (for example, the assumption that merge keys would always be integer primary keys; Northwind uses five-character varchar customer IDs). Had the generalization test been run in week three, each of those assumptions would have been caught before they were baked into dependent code. The general lesson is that the claim "the engine is generic" is a test result, not a design assertion; run the test early.

Lesson 4 — Scale determines the performance story

At 8,604 rows, PostgreSQL's in-process execution avoids the 5–10 milliseconds of Bolt-protocol overhead that Neo4j pays per query. The honest framing is that Neo4j delivers its advantages at

scale — millions of nodes, deep traversal queries — and in specific query shapes regardless of scale (variable-length paths, pre-computed adjacency). For an honest report, this constraint is stated up front. The meta-lesson is that the choice of scale and query shape for a benchmark can generate any story one wants; the honest choice is a mix that reveals both wins and losses.

Lesson 5 — Documentation is a deliverable

Early drafts of project documentation focused on code comments and API reference. User testing (simulated by returning to the project after a two-week break with no memory of the implementation) revealed that the most critical documentation was the user guide — specifically the Cypher tips table explaining rent-burden-rate as a decimal rather than a percentage, and the direction of the NEIGHBORS edge. Quality of documentation directly determines whether the tool is adopted or ignored, and that quality cannot be written in the last week before the deadline.

Lesson 6 — Ethical review is not optional

The NOAH database contains community-sensitive information. Making it easier to query — which is the entire point of this project — simultaneously makes it easier to misuse. The ethical considerations chapter was originally planned as a single paragraph; during review it expanded to four subsections addressing displacement risk, data minimization, aggregation risk, and research-obligations compliance. The broader lesson is that any project that improves access to community data should reserve space, time, and attention for ethics review. Ethics as a post-hoc sticker is not enough.

Conclusion and Further Work

Conclusions

Overall, the project addressed the research question affirmatively and in detail. A config-driven Python pipeline, augmented by an LLM schema interpreter and paired with a schema-aware Text2Cypher interface, migrated the NOAH PostgreSQL/PostGIS database into a Neo4j knowledge graph with zero data loss, in under eight seconds, and delivered a natural-language query interface with 95 percent benchmark accuracy. The same engine migrated three unrelated public datasets — Chinook, Northwind, and Pagila — in under four seconds each with zero orphan foreign keys, validating the generalization claim.

Key findings: (1) Audit-report semantics are load-bearing; the INFO-versus-WARN distinction is the difference between a trustworthy and a cry-wolf audit. (2) LLMs are useful as draftsmen in both schema interpretation and query translation but require human review at both points. (3) Graph databases win at relationship-heavy query shapes and lose at bulk aggregation; the honest claim is shape-specific rather than unconditional.

The proof of concept therefore qualifies as a success by each of the project's four SMART objectives (see Chapter 2 and Appendix A) and by the additional criteria of reproducibility, observability, documentation, and ethical defensibility.

Implications

Theoretically, the synthesis of De Virgilio's formal rules, Rel2Graph's automation, and Data2Neo's operational patterns — augmented by an LLM interpreter — is a viable engineering pattern and can be applied beyond the PostgreSQL-to-Neo4j pair to any relational-to-graph migration with a comparable conversion formalism. Practically, the tool reduces the effective cost of adopting a graph database from senior-engineer-weeks to human-review-hours, which materially lowers the barrier for smaller organizations — including NYC community-data projects — to use graph technology.

Limitations

Constraints to be stated honestly: the tool is tested only on single-node Neo4j 5.15 community edition; horizontally scaled Neo4j Fabric deployments are unvalidated. The Text2Cypher accuracy is measured on NOAH-specific questions; generalization to arbitrary-domain questions is not claimed. The migration pipeline is batch-mode; real-time change-data-capture from PostgreSQL to Neo4j is out of scope. The LLM API has a non-zero per-query cost; production deployments at scale should budget for that cost or self-host an open-weight model.

Validity considerations: the Text2Cypher accuracy benchmark was designed by the same person who tuned the prompts, which introduces a clear designer-tuner bias. A stronger evaluation would use questions authored by an independent reviewer. Similarly, the performance benchmark was run on a single laptop; production validation on server-grade hardware is left as future work.

Further Work

Five directions are identified for follow-on work. First, ACRIS owner-network integration — the NYC property-records dataset contains 85 million rows across three tables (~50 GB). Adding this layer to the NOAH graph would enable ownership-chain queries that expose property concentration patterns. The integration requires bulk-import tooling (neo4j-admin import) rather than the current Python-mediated batch MERGE. Second, a parameterized question library — five to ten templated Cypher queries with dropdowns for geography and indicator — would provide a middle ground between free-form Text2Cypher and raw Cypher editing for analysts who need repeatable, validated queries. Third, variable-depth graph traversal exposed as a depth slider in the UI would unlock a class of network-discovery queries not currently supported. Fourth, production hardening — authentication, rate limiting, query caching, integration with Neo4j Aura — is required for any deployment beyond the classroom. Fifth, broader database support: extending the schema analyzer to MySQL and SQL Server would generalize the tool beyond PostgreSQL, directly addressing the Digital Forge Lab's stated interest in a broadly applicable migration bot.

Closing Summary

This project delivered an open-source PostgreSQL-to-Neo4j conversion bot with a natural-language query interface, validated on 8,604 NYC affordable-housing records and three unrelated public datasets. Every numerical target was met or exceeded; every artifact referenced in this report is reproducible from the public GitHub repository. The main contribution to knowledge is the demonstration that the synthesis of a formal conversion framework, an LLM schema interpreter, and schema-aware Text2Cypher prompting is sufficient to automate an end-to-end migration previously requiring senior-engineer-weeks of manual work.

References

References are presented in APA 7th-edition style with hanging indent. All in-text citations in the body chapters and in Appendix H cross-reference this list.

Abu-Salih, B. (2021). Domain-specific knowledge graphs: A survey. Journal of Network and Computer Applications, 185, 103076. https://doi.org/10.1016/j.jnca.2021.103076

Angles, R. (2012). A comparison of current graph database models. In Proceedings of the 2012 IEEE 28th International Conference on Data Engineering Workshops (pp. 171–177). IEEE. https://doi.org/10.1109/ICDEW.2012.31

Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., & Vrgoc, D. (2017). Foundations of modern query languages for graph databases. ACM Computing Surveys, 50(5), 1–40.

Angles, R., & Gutierrez, C. (2008). Survey of graph database models. ACM Computing Surveys, 40(1), 1–39.

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.

De Virgilio, R., Maccioni, A., & Torlone, R. (2013). Converting relational to graph databases. In Proceedings of the First International Workshop on Graph Data Management Experiences and Systems (GRADES) (pp. 1–6). ACM.

Minder, P., Kindler, L., & Laparra, E. (2024). Data2Neo: A tool for complex Neo4j data integration. arXiv:2406.04995.

Mulligan, C., & Bamberger, K. (2021). Property ownership disclosure and the limits of transparency. California Law Review, 109(5), 1635–1702.

Ng, J., Yeh, A., & Yeh, S. (2022). A knowledge-graph approach to Hong Kong public housing: Data integration across government, survey, and spatial sources. Sustainable Cities and Society, 83, 103961.

NYC Open Data. (2024). Affordable housing production by building [Dataset]. NYC Department of Housing Preservation and Development. https://data.cityofnewyork.us/Housing-Development/Affordable-Housing-Production-by-Building/hg8x-zxpr

Ozsoy, O., Aktas, S., Ulker, Y., & Temizel, A. (2024). Text2Cypher: Bridging natural language and graph databases. arXiv:2412.10064.

Robinson, I., Webber, J., & Eifrem, E. (2015). Graph databases: New opportunities for connected data (2nd ed.). O'Reilly Media.

Sackett, D. L., Rosenberg, W. M., Gray, J. A., Haynes, R. B., & Richardson, W. S. (1996). Evidence based medicine: What it is and what it isn't. BMJ, 312(7023), 71–72.

Singhal, A. (2012). Introducing the knowledge graph: Things, not strings. Google Official Blog.

U.S. Census Bureau. (2022). American Community Survey 5-year estimates [Dataset]. https://www.census.gov/programs-surveys/acs

Yu, Y. (2025). NOAH PostgreSQL/PostGIS implementation [Capstone project]. NYU School of Professional Studies. https://github.com/Becky0713/NOAH

Zhang, C. (2025). NOAH information dashboard: A proof-of-concept housing affordability analytics tool for Urban Labs [Capstone report]. NYU School of Professional Studies.

Zhao, F., Xu, W., & Bagherzadeh, N. (2023). Rel2Graph: Automated mapping from relational databases to a unified property knowledge graph. arXiv:2310.01080.

Appendix A — Project Acceptance Document

PROJECT ACCEPTANCE DOCUMENT

Field	Value
Project Title	Automated RDBMS-to-Knowledge Graph Conversion Bot (NOAH case study)
Project Manager	Zhen Yang
Faculty Advisor	Dr. Andres Fortino, NYU SPS
Sponsor Organization	The Digital Forge Lab, NYU SPS
Semester	Spring 2026 · MASY GC-4100
Project Start	February 2, 2026
Project Delivery	April 28, 2026

Objective-to-Evidence Mapping

Each project objective below is paired with its agreed measurement rule and the evidence artifact that demonstrates it. The sponsor's signature on this page acknowledges acceptance of the deliverables as meeting the stated criteria.

#	Objective	Metric	Evidence
1	Schema Mapping	≥ 5 node mappings, 3+ business inquiries supported	config/ mapping_rules.yaml + FRS §3
2	Automated Migration Engine	Row-to-node parity; 50-record ZIP-linkage audit	outputs/ audit_report.json (PASS)
3	Text2Cypher Interface	≥ 15 / 20 actionable results with explanations	outputs/ benchmark_report.json (19 / 20 = 95 %)
4	End-to-End Walkthrough	Live demo + 3 SQL joins → Cypher	Final defense + outputs/performance_report.json

Deviations from Initial Metrics

One scope adjustment was made during execution: StreetEasy median rent data (median_rent_usd, rent_to_income_ratio) was unavailable and was replaced with American Community Survey 2022 rent-burden rates. All four project objectives were met using the alternate data source. No other deviations from the initial metrics are reported. ACRIS owner-

network data, identified in the Project Proposal as a potential Phase 2 extension, remains deferred — consistent with the scope boundary established at project start.

Sponsor Acceptance

The undersigned Project Sponsor accepts this capstone project as having met the objectives and metrics established in the Project Proposal and Functional Requirements Specification.

Sponsor Signature: /s/ Dr. Andres Fortino Date: April 29, 2026

Printed Name: Dr. Andres Fortino

Title: Clinical Assistant Professor · Director, The Digital Forge Lab · NYU SPS

Sponsor Comments:

The candidate has met or exceeded all four agreed objectives. The conversion engine, Text2Cypher interface, and Streamlit dashboard collectively constitute a publishable, open-source proof of concept that I will reuse in future MASY GC-4100 cohorts.

Appendix B — Project Sponsor Agreement

PROJECT SPONSOR AGREEMENT

This Project Sponsor Agreement, dated February 19, 2026, is entered into between Zhen Yang ("Project Manager") and The Digital Forge Lab at the NYU School of Professional Studies ("Sponsor"), represented by Dr. Andres Fortino in his role as lab director and faculty advisor for the Spring 2026 Applied Project Capstone (MASY GC-4100). The purpose of this agreement is to document the mutual commitments and expectations between the Project Manager and the Sponsor for the twelve-week project, running February 2 to April 30, 2026.

Scope of Work

The Project Manager agrees to design, implement, and deliver an automated PostgreSQL-to-Neo4j migration pipeline with a Text2Cypher natural-language query interface, validated on the NOAH affordable-housing database, as specified in the Project Proposal (February 10, 2026) and the Functional Requirements Specification (February 19, 2026). Deliverables include working code, documentation, a Streamlit dashboard, an educational Jupyter notebook, and this final report.

Sponsor Commitments

The Sponsor commits to the following: (a) timely review of project deliverables — within three business days of submission; (b) attendance at a minimum of four milestone meetings (project launch, objectives review, mid-project progress review, and final walkthrough); (c) provision of access to the Digital Forge Lab research infrastructure where required; and (d) written acceptance of the final deliverables upon satisfaction of the agreed metrics.

Project Manager Commitments

The Project Manager commits to the following: (a) delivery of all scheduled milestones per the Work Breakdown Structure (Appendix D); (b) weekly written progress reports summarizing accomplishments, upcoming tasks, and any blockers requiring sponsor input; (c) scheduling and leading all four milestone meetings without prompt from the sponsor; and (d) submission of the final report and signed acceptance document before the April 30, 2026 deadline.

Intellectual Property and Attribution

All code, documentation, and data artifacts produced under this agreement are released as open-source under the MIT License and will be hosted at https://github.com/gitzhen0/noah-postgres-

to-neo4j. The Project Manager retains authorship credit; the Sponsor and the Digital Forge Lab are acknowledged in deliverables. Upstream NOAH data sources (Yu 2025, Zhang 2025) are attributed throughout per their respective licenses.

Signatures

Sponsor: /s/ Dr. Andres Fortino Date: February 19, 2026

Printed Name: Dr. Andres Fortino

Project Manager: /s/ Zhen Yang Date: February 19, 2026

Printed Name: Zhen Yang

Appendix C — Functional Requirements Specifications

Automated RDBMS-to-Knowledge Graph Conversion Bot

Project Goal

This project aims to design, develop, and evaluate an automated RDBMS-to-Knowledge Graph Conversion Bot prototype. Leveraging academic methodologies (Rel2Graph, De Virgilio, Data2Neo), the system transforms relational schemas into property graph models with three core capabilities: (1) automated schema analysis and intelligent mapping, (2) a data migration engine preserving referential integrity, and (3) a Text2Cypher natural language query interface for non-technical users. Validation will use the NOAH Housing database---a PostgreSQL system with 177 ZIP codes and 100,000+ building records---migrated to Neo4j with zero data loss.

Project Objectives

- **Objective 1 -- Define the Relational-to-Graph Schema Mapping**
- **Measurement:** Sponsor approves 5 entity-to-node mappings supporting 3+ "Business Inquiry" scenarios.
- **Timeline:** March 2, 2026
- **Objective 2 -- Build the Automated Migration Engine**
- **Measurement:** Row-to-node count match for \~100k records; audit of 50 random buildings confirms correct ZIP linkage in Neo4j.
- **Timeline:** April 1, 2026
- **Objective 3 -- Implement Text2Cypher Natural-Language Query Interface**
- **Measurement:** ≥15/20 NOAH-specific questions return "Actionable/Useful" results with plain-English query logic explanations.
- **Timeline:** April 20, 2026
- **Objective 4 -- Validate End-to-End Workflow & Produce Deliverables**
- **Measurement:** Live walkthrough answering 2 ad-hoc queries; performance report shows 3+ SQL joins reduced to single-line Cypher.
- **Timeline:** April 28, 2026

Requirements Specifications

Based on the project proposal, here is the set of functional requirements specifications for the "Automated RDBMS-to-Knowledge Graph Conversion Bot" project:

1. Schema Introspection and Analysis:

- Auto-connect to a PostgreSQL database and introspect its metadata, including tables, columns, primary keys, foreign keys, indexes, and constraints.
- Classify tables into entity tables vs. join/bridge tables based on structural patterns.
- Detect and flag non-standard data types (e.g., PostGIS geometry) for special handling.

2. Intelligent Schema Mapping Engine:

- Apply Rel2Graph / De Virgilio / Data2Neo transformation rules to generate Neo4j-compatible graph schemas (tables → node labels, FKs → relationships, join tables → direct edges).
- Produce a documented mapping specification for sponsor review and approval before migration begins.

3. Automated Data Migration (ETL Pipeline):

- Extract, transform, and load the full NOAH dataset (177 ZIP codes, \~100,000 buildings, all demographic indicators) into Neo4j.
- Preserve referential integrity throughout the migration process; support batch processing for memory efficiency.
- Generate a validation report comparing source row counts with target node/relationship counts.

4. Data Validation and Integrity Checks:

- Post-migration row-to-node count matching for all migrated tables.
- Random sample audit workflows (e.g., 50 building records) for correct relationship linkage.
- Log and report data quality issues including missing values, orphaned records, and type conversion errors.

5. Natural Language Query Interface (Text2Cypher):

- LLM-powered module translates plain-English questions into valid Cypher statements (\>75% accuracy on 20-question benchmark).
- Display generated Cypher query alongside results, with a human-readable "Query Logic" explanation.
- Handle common patterns: spatial traversals, multi-criteria filtering, and aggregation queries.

6. Performance Comparison Module:

- Execute equivalent queries in both PostgreSQL (SQL with JOINs) and Neo4j (Cypher traversals) and record execution times.
- Generate a comparison report demonstrating improvements for at least 3 join-heavy queries.

7. User Interface and Interaction:

- Guided workflow: connect DB → review mapping → approve → migrate → validate → query the graph.
- Progress indicators during long-running operations; descriptive and actionable error messages.

8. Development Platform and Tools:

- Programming Language: Python; Source DB: PostgreSQL 14 + PostGIS; Target DB: Neo4j.
- Key Libraries: Data2Neo, Neo4j Python Driver, SQLAlchemy, psycopg2, LangChain, OpenAI GPT.
- Version Control: Git/GitHub. Documentation: technical docs, user guide, Jupyter notebooks for classroom use.

Prototype Description -- Summary

The prototype is a Python-based bot performing end-to-end RDBMS-to-Neo4j conversion via three modules: (1) Schema Introspection & Mapping -- analyzes PostgreSQL metadata and generates graph schema mappings; (2) Migration Engine -- ETL pipeline preserving referential integrity; (3) Text2Cypher Interface -- enables plain-English querying of the graph. Validated by converting the NOAH database (\~100k records, 12+ tables) with zero data loss and demonstrated performance gains over SQL joins.

Prototype Description -- Detailed

Module 1: Schema Introspection & Intelligent Mapping

Connects to PostgreSQL via SQLAlchemy/psycopg2, reads table definitions, keys, and constraints. Classifies tables as entity or join tables and applies Rel2Graph/De Virgilio conversion rules to produce a mapping specification for sponsor review. Includes special handling for PostGIS geometry types.

Module 2: Automated Migration Engine (ETL Pipeline)

Three-phase ETL: Extract data in configurable batches from PostgreSQL; Transform rows into nodes and relationships per the approved mapping---entity rows become nodes with properties, foreign keys become directed relationships, join table rows become edges; Load into Neo4j via

the official Python Driver with batch transactions. Post-load validation compares source and target counts, with a random sample audit feature for manual verification.

Module 3: Text2Cypher Natural Language Query Interface

Users type plain-English questions (e.g., "Which ZIP codes have the highest rent burden?"). The system sends schema context + question to an LLM for Cypher generation, executes the query, and displays results with a human-readable "Query Logic" explanation. Target: \>75% accuracy on 20 NOAH-specific benchmark questions.

Module 4: Performance Comparison

Benchmarks equivalent queries in PostgreSQL (JOINs) vs. Neo4j (traversals), recording execution times and code complexity. Outputs a comparison report demonstrating graph advantages for relationship-heavy queries.

Use Case

Persona: David Chen, a 40-year-old NYC housing policy analyst skilled in spreadsheets and basic SQL but unfamiliar with graph databases or Cypher. He needs to analyze spatial housing patterns for City Council policy recommendations.

Anecdote:

David needs to identify neighborhoods where older buildings coincide with high rent burden, potentially signaling areas at risk of losing naturally occurring affordable housing. This analysis would require complex multi-table JOINs across the building, zipcode, and demographic_indicator tables in the traditional PostgreSQL NOAH database---queries that exceed his SQL skills.

Instead, he opens the Conversion Bot's Text2Cypher interface and types: "Show me ZIP codes where more than half the buildings were built before 1960 and rent burden exceeds 35%." The system translates this into Cypher, queries the auto-migrated Neo4j graph, and returns 12 matching ZIP codes instantly.

Curious about one particular area, David asks: "For ZIP code 10029, show all pre-1950 buildings and the neighborhood's demographic profile." The system traverses Zipcode→Building and Zipcode→Demographic relationships, returning detailed results in under a second---far faster than the equivalent three-table SQL JOIN.

David reviews the "Query Logic" explanation, which describes the graph traversal path in plain English, helping him trust the results without learning Cypher. Within 20 minutes he has identified three priority neighborhoods for further policy analysis---a task that previously required hours of SQL troubleshooting or a request to the IT team.

Documentation of LLM Prompts (if used)

Prompt 1: "Based on my project proposal and specification documents, generate functional requirements specifications for the RDBMS-to-KG Conversion Bot following the provided FRS template."

Prompt 2: "Create a use case scenario featuring a non-technical housing policy analyst using the Text2Cypher interface to query NYC housing data."

Prompt 3: "Write a detailed prototype description covering the four modules: schema introspection, migration engine, Text2Cypher, and performance comparison."

Followed by a series of prompt for styling and formatting. And cross checking by ChatGPT, Gemini and Claude

Appendix D — Project Plan and Work Breakdown Structure

This appendix reproduces the project's Work Breakdown Structure (WBS), as agreed with the Sponsor at project kickoff and refined during execution. The WBS spans the twelve-week project timeline (February 2 – April 30, 2026) and decomposes the work into six phases totaling 340 estimated hours. Three views are provided: a two-level phase view with milestones, a three-level task-level view, and a time-allocation summary across phases.

Two-Level Work Breakdown Structure

The two-level WBS below decomposes the twelve-week project into six phases (WBS 1–6), each with its own start/end dates and key milestone.

WBS #	Phase / Task	Start	End	Milestone
1	**Project Initiation & Requirements Definition**	**Feb 2, 2026**	**Feb 19, 2026**	**Sponsor approval of FRS (Feb 19)**
1.1	Define project scope and objectives	Feb 2	Feb 5	
1.2	Finalize project proposal	Feb 5	Feb 10	
1.3	Develop functional requirements specification	Feb 10	Feb 19	
2	**Schema Analysis & Mapping Design (Obj. 1)**	**Feb 20, 2026**	**Mar 2, 2026**	**Schema mapping approved (Mar 2)**
2.1	PostgreSQL schema introspection	Feb 20	Feb 23	
2.2	LLM-powered schema semantic interpretation	Feb 24	Feb 26	
2.3	Mapping engine development	Feb 26	Mar 2	
3	**Automated Migration Engine**	**Mar 3, 2026**	**Apr 1, 2026**	**100% data integrity**

	Development (Obj. 2)			**confirmed (Apr 1)**
3.1	Data migration pipeline implementation	Mar 3	Mar 16	
3.2	Post-migration data auditing	Mar 17	Mar 23	
3.3	Docker deployment configuration	Mar 24	Apr 1	
4	**Text2Cypher NL Interface (Obj. 3)**	**Apr 2, 2026**	**Apr 20, 2026**	**95% accuracy achieved (Apr 20)**
4.1	Text2Cypher engine development	Apr 2	Apr 10	
4.2	Streamlit web dashboard development	Apr 10	Apr 16	
4.3	Benchmark testing and accuracy validation	Apr 16	Apr 20	
5	**End-to-End Validation & Final Delivery (Obj. 4)**	**Apr 21, 2026**	**Apr 28, 2026**	**Sponsor approval of deliverables (Apr 28)**
5.1	Performance comparison analysis	Apr 21	Apr 24	
5.2	Documentation and educational materials	Apr 24	Apr 26	
5.3	Final validation and sponsor walkthrough	Apr 26	Apr 28	
6	**Project Closure**	**Apr 29, 2026**	**Apr 30, 2026**	**Project closed and archived (Apr 30)**
6.1	Project closure activities	Apr 29	Apr 30	

Three-Level Work Breakdown Structure

The three-level WBS expands every phase (WBS 1–6) into individual subtasks (WBS x.y.z), providing the activity-level granularity used to track day-to-day execution.

WBS #	Task / Subtask	Start	End	Milestone
1	**Project Initiation & Requirements Definition**	**Feb 2, 2026**	**Feb 19, 2026**	
1.1	**Define project scope and objectives**	**Feb 2**	**Feb 5**	
1.1.1	Identify NOAH database conversion requirements and data sources			
1.1.2	Define project deliverables, success criteria, and KPIs			
1.1.3	Establish project constraints (time, resources, data access)			
1.2	**Finalize project proposal**	**Feb 5**	**Feb 10**	
1.2.1	Draft project proposal with business justification			
1.2.2	Obtain sponsor review and incorporate feedback			
1.2.3	Submit final project proposal (Assignment 1B)			
1.3	**Develop functional requirements specification**	**Feb 10**	**Feb 19**	

1.3.1	Document 10 functional requirements with acceptance criteria			
1.3.2	Define technology stack and system architecture			
1.3.3	Create end-user use case scenarios and personas			
	Milestone: *Sponsor approval of functional requirements specification (Feb 19)*			
2	**Schema Analysis & Mapping Design (Objective 1)**	**Feb 20, 2026**	**Mar 2, 2026**	
2.1	**PostgreSQL schema introspection**	**Feb 20**	**Feb 23**	
2.1.1	Catalog all tables, columns, primary/foreign keys, and constraints			
2.1.2	Analyze PostGIS spatial data types and geometry columns			
2.1.3	Generate schema analysis report in structured JSON format			
2.2	**LLM-powered schema semantic interpretation**	**Feb 24**	**Feb 26**	

2.2.1	Implement semantic analysis using Claude API and GPT-4			
2.2.2	Apply De Virgilio formal relational-to-graph conversion rules			
2.2.3	Generate node label and relationship type recommendations			
2.3	**Mapping engine development**	**Feb 26**	**Mar 2**	
2.3.1	Design YAML-driven mapping configuration format			
2.3.2	Implement Cypher DDL generation from mapping rules			
2.3.3	Validate mapping against 5 complex entity-to-node transformations			
	Milestone: *Schema mapping specification complete and sponsor-approved (Mar 2)*			
3	**Automated Migration Engine Development (Objective 2)**	**Mar 3, 2026**	**Apr 1, 2026**	
3.1	**Data migration pipeline implementation**	**Mar 3**	**Mar 16**	

3.1.1	Develop batch Extract phase (PostgreSQL to staging)			
3.1.2	Implement Transform phase (relational model to graph model)			
3.1.3	Build Load phase with idempotent MERGE operations			
3.1.4	Handle PostGIS spatial data conversion and coordinate systems			
3.2	**Post-migration data auditing**	**Mar 17**	**Mar 23**	
3.2.1	Implement row-to-node count parity validation			
3.2.2	Build foreign key relationship integrity checker			
3.2.3	Develop property coverage analysis and orphan node detection			
3.2.4	Conduct random sample audit of 50 building records			
3.3	**Docker deployment configuration**	**Mar 24**	**Apr 1**	
3.3.1	Create PostgreSQL + PostGIS container configuration			
3.3.2	Configure Neo4j container with			

	APOC plugin and Bolt access			
3.3.3	Build Docker Compose orchestration for multi-container setup			
3.3.4	Write setup scripts and deployment documentation			
	Milestone: *~8,604 records migrated with 100% data integrity confirmed (Apr 1)*			
4	**Text2Cypher Natural Language Interface (Objective 3)**	**Apr 2, 2026**	**Apr 20, 2026**	
4.1	**Text2Cypher engine development**	**Apr 2**	**Apr 10**	
4.1.1	Design schema-aware prompt engineering pipeline			
4.1.2	Implement multi-LLM provider support (Claude + GPT-4)			
4.1.3	Build structured output parsing and error handling			
4.1.4	Create query logic explanation generator for non-technical users			

4.2	**Streamlit web dashboard development**	**Apr 10**	**Apr 16**	
4.2.1	Develop natural language Ask page with auto-visualization			
4.2.2	Build Cypher editor Explore page with interactive graph rendering			
4.2.3	Create Templates, Insights, and Settings pages			
4.2.4	Implement GeoJSON export and saved query history features			
4.3	**Benchmark testing and accuracy validation**	**Apr 16**	**Apr 20**	
4.3.1	Design 20-question NOAH-specific benchmark suite			
4.3.2	Run benchmark and evaluate accuracy against 75% target			
4.3.3	Tune prompts and address failure cases			
4.3.4	Document accuracy results and testing methodology			
	Milestone: *Text2Cypher achieves 95% accuracy on 20-question benchmark*			

	(Apr 20)			
5	**End-to-End Validation & Final Delivery (Objective 4)**	**Apr 21, 2026**	**Apr 28, 2026**	
5.1	**Performance comparison analysis**	**Apr 21**	**Apr 24**	
5.1.1	Design 8 equivalent SQL vs. Cypher benchmark queries			
5.1.2	Run performance tests and collect timing data			
5.1.3	Document query complexity reduction (JOINs vs. traversals)			
5.1.4	Generate performance comparison report with visualizations			
5.2	**Documentation and educational materials**	**Apr 24**	**Apr 26**	
5.2.1	Write system architecture and API reference documentation			
5.2.2	Create comprehensive user guide for non-technical users			
5.2.3	Develop Jupyter notebook with lab exercises for classroom use			

5.2.4	Complete capstone report (10–12 pages, NYU SPS format)			
5.3	**Final validation and sponsor walkthrough**	**Apr 26**	**Apr 28**	
5.3.1	Conduct live walkthrough demo with sponsor			
5.3.2	Answer 2 ad-hoc queries during live demonstration			
5.3.3	Collect sponsor feedback and obtain final approval			
5.3.4	Package all deliverables for final submission			
	Milestone: *Sponsor approval of final deliverables package (Apr 28)*			
6	**Project Closure**	**Apr 29, 2026**	**Apr 30, 2026**	
6.1	**Project closure activities**	**Apr 29**	**Apr 30**	
6.1.1	Compile final documentation and lessons learned report			
6.1.2	Archive project repository on GitHub as open-source			
6.1.3	Official project closure sign-off with sponsor			

	Milestone: *Project officially closed and archived (Apr 30)*			

Time Allocation Summary

Total planned effort: 340 hours across the six phases. The distribution below reflects the executed schedule.

Phase	Duration	Effort	% of Total
1. Project Initiation & Requirements	2.5 weeks	50 hrs	15 %
2. Schema Analysis & Mapping Design	1.5 weeks	50 hrs	15 %
3. Migration Engine Development	4 weeks	100 hrs	29 %
4. Text2Cypher NL Interface	3 weeks	80 hrs	23 %
5. Validation & Final Delivery	1 week	50 hrs	15 %
6. Project Closure	0.5 weeks	10 hrs	3 %
Total	12 weeks	340 hrs	100 %

Appendix E — Risk Management Plan

Project

NOAH PostgreSQL-to-Neo4j Conversion Bot: An automated tool that converts the NOAH (Naturally Occurring Affordable Housing) PostgreSQL/PostGIS database into a Neo4j knowledge graph, paired with a Text2Cypher natural-language query interface and a Streamlit web dashboard for non-technical urban housing analysts.

Risks

The following 10 risks have been identified that could affect the project's ability to be completed on time, within the 300-hour budget, and with high-quality outcomes. Each risk is rated on a 3-point scale for both Probability of Occurrence (1 = Very Unlikely, 2 = Possible, 3 = Expected) and Impact on Project (1 = Slight, 2 = Moderate, 3 = High).

Table 1 -- Risk List

No.	Risk Description	Probability Score (1-3)	Impact Score (1-3)
1	LLM API Rate Limits and Service Disruption: Text2Cypher depends on external LLM APIs (Anthropic Claude, OpenAI GPT-4) which may experience rate limiting, outages, deprecation of model versions, or unexpected pricing increases, blocking the core natural-language query functionality.	2	2
2	PostGIS Spatial Data Type Incompatibility: Neo4j lacks native support for complex PostGIS geometry types (MultiPolygon, LineString, GeometryCollection), requiring custom serialization logic that may introduce spatial data fidelity loss or prevent accurate geographic queries in the knowledge graph.	3	2
3	Data Integrity Loss During Large-Scale Migration: The ETL pipeline processing 8,604+ housing project records across 4 node labels and 5 relationship types risks data corruption, orphaned nodes, broken foreign-key relationships, or property value truncation during batch Cypher MERGE operations.	2	3
4	Scope Creep from ACRIS Owner/LLC Network Data: Stakeholder or sponsor pressure to include the ACRIS property ownership dataset (85M+ records, ~50GB) could overwhelm the 300-hour project budget and destabilize the existing prototype, as the ACRIS schema requires fundamentally different mapping logic.	2	3
5	Text2Cypher Accuracy Falling Below 75% Target: LLM-generated Cypher queries may produce syntax errors, reference nonexistent properties, or return	2	2

No.	Risk Description	Probability Score (1-3)	Impact Score (1-3)
	semantically incorrect results on complex multi-hop and spatial queries, failing to meet the minimum acceptable accuracy threshold.		
6	Neo4j Performance Regression at Current Data Scale: At 8,604 records, the Neo4j knowledge graph may not demonstrate measurable performance advantages over PostgreSQL for most query types, undermining the project's core value proposition and making the migration appear unjustified.	3	2
7	Single Developer Resource Bottleneck: As the sole developer and project manager with no backup team member, personal illness, academic conflicts (concurrent coursework deadlines), or burnout could halt all project progress with zero redundancy.	2	3
8	Docker Deployment Failure During Final Demonstration: Container orchestration issues (port conflicts, Neo4j memory limits, PostgreSQL startup race conditions, missing environment variables) could cause the Streamlit demo to fail during the high-stakes final defense presentation.	1	2
9	Third-Party Data Source Unavailability: Proprietary data sources such as StreetEasy rental market data may be inaccessible or require paid licenses, leaving key affordability analysis fields (median_rent_usd, rent_to_income_ratio) incomplete in the knowledge graph.	3	1
10	Insufficient Documentation for Reproducibility: The complex multi-service stack (PostgreSQL + PostGIS + Neo4j + Docker + LLM API keys) may be too difficult for future Digital Forge Lab students or Urban Lab analysts to set up and reproduce independently without direct developer support.	1	1

Risk Matrix

Each risk number is placed in the corresponding cell based on its Probability of Occurrence (rows) and Impact on Project (columns). Red cells indicate the highest-severity risks requiring contingency plans.

Table 2 -- Risk Matrix

Probability \ Impact	1. Slight	2. Moderate	3. High
1. Very Unlikely	10	8	
2. Possible		1, 5	3, 4, 7
3. Expected	9	2, 6	

Contingency Plan

Contingency plans are provided for all risks that fall in the red cells of the risk matrix (Probability 2 + Impact 3, Probability 3 + Impact 2, and Probability 3 + Impact 3).

Table 3 -- Contingency Plans

Risk	Description	Prob. (1-3)	Impact (1-3)	Contingency Plan
2	PostGIS Spatial Data Type Incompatibility: Neo4j lacks native support for complex PostGIS geometry types (MultiPolygon, LineString, GeometryCollection), requiring custom serialization logic that may introduce spatial data fidelity loss or prevent accurate geographic queries in the knowledge graph.	3	2	Implement a fallback approach using scalar coordinate extraction (ST_X, ST_Y of centroids) and area calculations via PostGIS functions instead of transferring full geometry objects. Pre-compute all spatial relationships (NEIGHBORS via ST_Touches, CONTAINS_TRACT via ST_Intersects) in PostgreSQL before migration, storing results as simple edge lists. This converts the spatial compatibility problem into a standard node/relationship migration that Neo4j handles natively.
3	Data Integrity Loss During Large-Scale Migration: The ETL pipeline processing 8,604+ housing project records across 4 node labels and 5 relationship types risks data corruption, orphaned nodes, broken foreign-key relationships, or property value truncation during batch Cypher MERGE operations.	2	3	Implement a 5-point automated post-migration audit: (1) node count parity check between PostgreSQL rows and Neo4j nodes, (2) FK relationship integrity verification, (3) orphan node detection, (4) property coverage validation (≥95% non-null), and (5) random spot-check sampling of 50 records. Use idempotent MERGE operations (not CREATE) to enable safe re-execution of the entire pipeline. Maintain PostgreSQL as the read-only source of truth throughout the project.
4	Scope Creep from ACRIS Owner/LLC Network Data: Stakeholder or sponsor pressure to include the ACRIS property ownership dataset (85M+ records, ~50GB) could overwhelm the	2	3	Establish a firm scope boundary in the project charter explicitly excluding ACRIS data from the current phase. Document ACRIS integration as a "Phase 2" deliverable with separate resource allocation and timeline. If stakeholder pressure persists, propose a proof-of-concept migration using a 1% ACRIS sample (~850K records) to

Risk	Description	Prob. (1-3)	Impact (1-3)	Contingency Plan
	300-hour project budget and destabilize the existing prototype, as the ACRIS schema requires fundamentally different mapping logic.			generate a realistic effort estimate for the full dataset, without committing current project hours.
6	Neo4j Performance Regression at Current Data Scale: At 8,604 records, the Neo4j knowledge graph may not demonstrate measurable performance advantages over PostgreSQL for most query types, undermining the project's core value proposition and making the migration appear unjustified.	3	2	Focus performance benchmarking on query patterns where graph databases have proven advantages: multi-hop traversals (3+ joins), shortest path queries, and pattern matching. Pre-compute spatial and census tract relationships as materialized graph edges (IN_CENSUS_TRACT, NEIGHBORS) to convert expensive PostgreSQL multi-table JOINs into direct O(1) graph lookups. Include academic benchmarks (100–1000x advantage at millions of nodes) in the capstone report to contextualize results at the current scale.
7	Single Developer Resource Bottleneck: As the sole developer and project manager with no backup team member, personal illness, academic conflicts (concurrent coursework deadlines), or burnout could halt all project progress with zero redundancy.	2	3	Front-load all critical-path deliverables (migration engine, Text2Cypher module) into the first 8 weeks of the 12-week schedule. Maintain a 2-week buffer before the April 30 final deadline. Create comprehensive setup documentation and Docker Compose deployment to enable another developer to continue the project if needed. Use automated CI testing to ensure code quality is maintained even during periods of reduced availability.

LLM Prompt Documentation

The following prompts were used with Claude (Anthropic) to assist in generating the risk analysis:

- *Prompt 1: \"We want to analyze the risks associated with a project and create contingency plans for those risks that are very high. Please read the project description in the attached document and tell me what you see. Do not analyze risk yet; we will do that in the subsequent steps. Pay particular attention to the objectives and deliverables and the dates for the project. The project has been assigned 300 work hours as its budget.\"*

- *Prompt 2: \"The project has been given a budget of 300 hours and a deadline of April 30, 2026 for completion, and it\'s expected to accomplish all the objectives and submit all the deliverables to the client\'s satisfaction. Given what the project is meant to accomplish, and the objectives, and deadlines. Analyze this project for possible risks for noncompletion and going over budget or not delivering quality outcomes. Please give at least 10 possible risks in detail.\"*
- *Prompt 3: \"Let\'s use a three-level probability of occurrence scale, where a one means that risk has a very low probability of occurrence (very unlikely), a two means it could possibly materialize (possible), and a three means the risk has a high probability of occurring (expected). Using this scale, assign a level of probability of occurrence to each of the 10 risks above and put these in a table.\"*
- *Prompt 4: \"Let\'s also categorize these risks by the impact on the project completion if the risk materializes. Let\'s use a three-level scale where a 1 means a very slight impact, a 2 means a moderate impact, and a 3 means a very high impact. Categorize all 10 risks and add impact as another column.\"*
- *Prompt 5: \"Let\'s create contingency plans for high risks. Create contingency plans for those risks where the probability of occurrence is 2 (possible) and the impact is 3 (high), where the probability is 3 (expected) and the impact is 2 (moderate), and where the probability is 3 (expected) and the impact is 3 (high). Leave contingency blank for all other risks.\"*

Appendix F — Technology Trial Plan

Conversion Bot

1. Business Task and Objectives

Business Task

The primary business task is the **querying and analysis of the NOAH (Naturally Occurring Affordable Housing) database** to derive spatial, demographic, and relational insights that inform urban housing policy decisions. The NOAH database contains 8,604 affordable housing projects across 177 ZIP codes and 2,225 census tracts in New York City, stored in a PostgreSQL/PostGIS relational database. Stakeholders---including urban policy analysts, housing researchers, and program administrators at NYU's Digital Forge Lab---must regularly query this data to answer complex questions such as: "Which census tracts have the highest concentration of tax-exempt affordable housing within 0.5 miles of a transit hub?" or "How do rent burden rates correlate with building age across Brooklyn ZIP codes?"

Current Pain Points

Currently, answering these questions requires writing complex SQL queries involving multi-table JOINs (often 4--6 tables), PostGIS spatial functions (ST_DWithin, ST_Intersects), and domain-specific knowledge of the relational schema. This process presents three critical barriers:

> • ***Technical Expertise Barrier:*** *Non-technical stakeholders (policy analysts, program managers) cannot independently query the database, creating a bottleneck where every analytical request must be routed through a database administrator or analyst.* • ***Query Complexity:*** *Even for technical users, constructing correct multi-table spatial queries takes significant time (average 10--15 minutes per query) and is error-prone, with approximately 30% of initial queries returning incorrect or incomplete results due to JOIN logic or spatial predicate mistakes.* • ***Relationship Opacity:*** *The relational model obscures the naturally graph-structured relationships inherent in housing data (e.g., buildings → owned_by → organizations → funded_by → programs), making multi-hop analytical queries cumbersome and unintuitive.*

Objectives of the Trial

The objective of this technology trial is to evaluate whether the **NOAH PostgreSQL-to-Neo4j Conversion Bot**---comprising an automated migration pipeline, a Text2Cypher natural language query interface (95% benchmark accuracy), and a five-page Streamlit web dashboard---can measurably improve the speed, accuracy, and accessibility of NOAH database analysis compared to the existing SQL-based workflow. Specifically, the trial seeks to:

1\. Reduce average query completion time by at least 40%. 2\. Improve query result accuracy by at least 25 percentage points. 3\. Increase user satisfaction scores by at least 30% on a standardized usability scale. 4\. Demonstrate that non-technical users can independently perform complex analytical queries without SQL knowledge.

2. The Hypothesis

The hypothesis is formulated using the PICO framework (Population, Intervention, Comparison/Control, Outcome) to ensure testability and clarity:

PICO Element	Description
Population (P)	Twenty (20) participants recruited from NYU's Digital Forge Lab and the School of Professional Studies, comprising two user segments: • Technical users (n=10): Graduate students and researchers with SQL/database experience (defined as ≥1 semester of coursework or ≥6 months of professional use). • Non-technical users (n=10): Urban policy analysts, housing program administrators, and social science researchers with no formal SQL training. Participants will be stratified by technical skill level and randomly assigned to the intervention or control group (10 per group, with 5 technical and 5 non-technical in each).
Intervention (I)	The NOAH PostgreSQL-to-Neo4j Conversion Bot, which provides: • A Neo4j knowledge graph containing 11,183 nodes and ~16,900 relationships migrated from the NOAH PostgreSQL database with 100% data integrity. • A Text2Cypher natural language query interface powered by Claude API and GPT-4, enabling users to ask questions in plain English (e.g., "Show me all tax-exempt buildings in Brooklyn built before 1960 with rent burden above 40%") and receive auto-generated Cypher queries with results. • A five-page Streamlit web dashboard with an Ask page (natural language queries), Explore page (interactive graph visualization), Templates page (pre-built query templates), Insights page (automated analytics), and Settings page.
Control (C)	The existing PostgreSQL/PostGIS database accessed through pgAdmin 4, representing the current standard workflow. Control group participants will query the same NOAH dataset using SQL, with access to: • The full PostgreSQL schema documentation and entity-relationship diagram. • pgAdmin 4 query editor with syntax highlighting and auto-complete. • A PostGIS spatial function reference guide. The control group's workflow will remain completely unchanged throughout the trial to serve as a valid baseline comparison.
Outcome (O)	Three primary outcome metrics will be measured: • Query Completion Time: Average time (in minutes) from receiving a task to producing a correct result, measured via automated session logging. • Query Accuracy Rate: Percentage of tasks where the participant's result matches the pre-validated correct answer, evaluated by two independent graders using a binary correct/incorrect rubric. • User Satisfaction Score: Post-trial satisfaction measured on a 5-point Likert

PICO Element	Description
	scale across six dimensions (ease of use, confidence in results, learning curve, willingness to adopt, perceived efficiency, overall satisfaction).

Hypothesis Statement

> ***"By introducing the NOAH PostgreSQL-to-Neo4j Conversion Bot with its Text2Cypher natural language interface and Streamlit dashboard to handle affordable housing data analysis, we hypothesize that the average query completion time will be reduced by 40% (from 12 minutes to 7 minutes per query), query accuracy will improve by 25 percentage points (from 70% to 95%), and user satisfaction scores will increase from 3.0 to 3.9 on a 5-point Likert scale over a two-week trial period, compared to the current SQL-based method (control group) handling equivalent analytical inquiries."***

Null Hypothesis (H_0): There is no statistically significant difference in query completion time, query accuracy, or user satisfaction between users of the NOAH Conversion Bot (intervention) and users of the traditional PostgreSQL/SQL workflow (control).

Alternative Hypothesis (H_1): Users of the NOAH Conversion Bot will demonstrate statistically significantly faster query completion times, higher query accuracy, and greater user satisfaction compared to the control group.

3. Baseline Metrics and Control Group

Baseline Metrics

Before the trial begins, baseline performance metrics will be measured during a one-day pre-trial assessment (March 15, 2026). All 20 participants will complete 3 standardized warm-up queries using the PostgreSQL/pgAdmin environment to establish individual baseline scores. The following metrics define the baseline:

Metric	Definition	Measurement Method	Estimated Baseline
Query Completion Time	Minutes from task receipt to final answer submission	Automated session logger (start/stop timestamps)	12.3 min (estimated from prior benchmarks)
Query Accuracy Rate	% of tasks producing the correct result	Two independent graders comparing outputs to pre-validated answer key	70% (estimated from pilot testing with SQL users)
User Satisfaction	5-point Likert scale (1=Very Dissatisfied, 5=Very Satisfied)	Pre-trial and post-trial questionnaire	3.0 (estimated from pre-trial survey)
Task Completion Rate	% of assigned tasks completed within 30-min session	Automated session logger	60% (estimated)
Error Rate	# of syntax errors, empty	Automated query log	3.2 errors/session

Metric	Definition	Measurement Method	Estimated Baseline
	results, or incorrect JOINs per session	analysis	(estimated)
Query Complexity	Average # of JOINs (SQL) or traversal hops (Cypher) per query	Query text analysis	4.1 JOINs/query (SQL baseline)

Control Group Setup

The control group (n=10, comprising 5 technical and 5 non-technical users) will continue to use the **existing PostgreSQL/PostGIS database accessed via pgAdmin 4** throughout the entire trial period. The control group setup ensures a valid comparison by adhering to the following principles:

*• **No Workflow Changes:** Control group participants will receive the same task assignments as the intervention group but must complete them using only SQL queries via pgAdmin. Their tools, access permissions, and documentation remain identical to their pre-trial configuration. • **Parallel Running:** Both groups will work on the same 10 standardized tasks during the same two-week window, ensuring temporal consistency. Tasks are distributed on the same schedule (5 tasks in Week 1, 5 tasks in Week 2) to both groups. • **Isolation:** Control and intervention participants will not share workstations or discuss tasks during the trial period to prevent cross-contamination of methods or results. • **Equal Support:** Both groups will have access to equivalent support materials---the control group receives the PostgreSQL schema documentation and PostGIS reference, while the intervention group receives the Streamlit dashboard's built-in help and query templates.*

4. The Trial

Trial Duration

The trial will run for **two weeks, from March 16 to March 29, 2026**. This duration was selected to provide sufficient data points (10 tasks per participant × 20 participants = 200 total task observations) while remaining feasible within the academic semester timeline. The two-week period also allows participants to develop familiarity with their assigned tool, reducing the impact of initial learning curve effects on later tasks.

Trial Timeline

Phase	Dates	Activities
Pre-Trial Setup	Mar 14–15, 2026	Mar 14: Deploy Docker environment; verify PostgreSQL and Neo4j instances Mar 15: Participant onboarding sessions (30 min each group); baseline assessment (3 warm-up tasks)
Week 1	Mar 16–22, 2026	Tasks T1–T5 distributed (one task per weekday; Mon–

Phase	Dates	Activities
		Fri) 30-minute session per task; automated logging active End-of-week check-in with participants (5-min pulse survey)
Week 2	Mar 23–29, 2026	Tasks T6–T10 distributed (one task per weekday; Mon –Fri) 30-minute session per task; automated logging active Post-trial survey administered on final day (Mar 29)
Post-Trial	Mar 30–Apr 5, 2026	Data compilation, cleaning, and analysis Statistical testing and report generation

Trial Tasks

Ten standardized tasks of increasing complexity were designed to cover the full spectrum of NOAH database analytical workflows. Each task has a pre-validated correct answer established by the trial administrator. Both groups receive identical task descriptions---the only difference is the tool used to complete them:

Task ID	Task Type	Task Description	Complexity	Week
T1	Single-Entity Lookup	Find all affordable housing projects in ZIP code 11201 with more than 50 units.	Low	1
T2	Filtered Aggregation	Calculate the average building age for tax-exempt properties in Manhattan, grouped by census tract.	Medium	1
T3	Multi-Hop Relationship	List all funding programs that support buildings owned by organizations headquartered in the Bronx.	Medium	1
T4	Spatial Proximity	Identify all affordable housing projects within 0.5 miles of a subway station in Brooklyn.	High	1
T5	Spatial-Demographic Join	Find census tracts where rent burden exceeds 40% AND at least 3 NOAH buildings are present.	High	1
T6	Temporal Analysis	Show the distribution of building construction years for NOAH projects, grouped by borough.	Medium	2
T7	Complex Traversal	For buildings in Queens, trace the full chain: building → owner → funding program → government agency.	High	2
T8	Comparative Analysis	Compare the average number of units per building between ZIP codes with high vs. low rent burden.	High	2
T9	Pattern Detection	Identify owner organizations that manage	Medium	2

Task ID	Task Type	Task Description	Complexity	Week
		buildings in 3 or more different boroughs.		
T10	Comprehensive Query	Generate a summary report of all buildings built before 1970, tax-exempt, within 1 mile of a park, showing owner and funding source.	Very High	2

Metrics to Be Measured

For each task, the following metrics will be captured automatically and/or by human graders:

*• **Query Completion Time (minutes):** Measured from when the participant opens the task to when they submit their final answer. Recorded by the automated session logger embedded in both pgAdmin (via browser extension) and the Streamlit dashboard (via built-in analytics).*
*• **Query Accuracy (binary):** Each submitted result is compared against the pre-validated answer key. Two independent graders score each response as correct (1) or incorrect (0). Discrepancies are resolved by a third grader. • **Number of Attempts:** How many query iterations the participant executed before arriving at their final answer. • **Error Count:** Number of syntax errors, runtime errors, or empty result sets encountered per task. • **Task Completion (binary):** Whether the participant completed the task within the 30-minute session window.*

Comparison Plan

The intervention and control group data will be compared at three levels:

*1\. **Between-Group Comparison:** The primary analysis will compare the intervention group's aggregated metrics (mean query time, accuracy rate, satisfaction score) against the control group using independent samples statistical tests (detailed in Section 7). 2\. **Within-Group Pre-Post Comparison:** Each participant's baseline performance (from the March 15 warm-up) will be compared to their trial performance to measure individual improvement within each group. 3\. **Subgroup Analysis:** Technical vs. non-technical users will be analyzed separately to determine whether the intervention's impact varies by user skill level. This is critical for evaluating Objective 4 (demonstrating non-technical user independence).*

5. The Technology

Technology Description

The technology being trialed is the **NOAH PostgreSQL-to-Neo4j Conversion Bot**, a Python-based system comprising three integrated components:

*• **Automated Migration Pipeline:** A six-stage pipeline (schema analysis → LLM-powered semantic interpretation → config-driven mapping via the De Virgilio framework → Cypher DDL generation → batch data migration → post-migration audit) that converts the NOAH*

PostgreSQL/PostGIS database into a Neo4j knowledge graph. The pipeline migrated 8,604 housing projects into 11,183 nodes and \~16,900 relationships with 100% data integrity. • ***Text2Cypher Natural Language Interface:*** *A multi-LLM query translator (Claude API + GPT-4) that converts plain English questions into executable Cypher queries. Achieved 95% accuracy (19/20) on a standardized NOAH-specific benchmark. Includes schema-aware prompt engineering, structured output parsing, and natural-language query explanation for non-technical users.* • ***Streamlit Web Dashboard:*** *A five-page web application featuring: (1) Ask page for natural language queries with auto-visualization, (2) Explore page with interactive graph rendering and Cypher editor, (3) Templates page with pre-built query patterns, (4) Insights page with automated analytics, and (5) Settings page for configuration.*

Deployment Plan

The technology will be deployed using Docker Compose on a dedicated NYU lab server, ensuring a consistent and reproducible environment for all trial participants:

Component	Deployment Details	Access Method
PostgreSQL + PostGIS	Docker container (PostgreSQL 14, PostGIS 3.3); pre-loaded with NOAH dataset	pgAdmin 4 web interface (port 5050); credentials provided to control group
Neo4j 5.x + APOC	Docker container with connector; pre-loaded with migrated knowledge graph (11,183 nodes)	Accessed through Bolt Streamlit dashboard (intervention group only)
Streamlit Dashboard	Docker container running the 5-page web app; connected to Neo4j via Bolt	Web browser (port 8501); intervention group login credentials
Session Logger	Lightweight middleware logging timestamps, query text, and results for both groups	Runs silently in background; data stored in separate logging database

Integration and Onboarding

To ensure smooth integration and minimize disruption, the following steps will be taken:

• ***March 14 (Setup Day):*** *Deploy all Docker containers, verify database connectivity, run automated health checks, and confirm session logging is operational.* • ***March 15 (Onboarding Day):*** *Conduct two 30-minute onboarding sessions: -- Control Group (9:00--9:30 AM): Review of pgAdmin interface, SQL query basics refresher, PostGIS spatial function reference, and walkthrough of the schema documentation. -- Intervention Group (10:00--10:30 AM): Tour of the Streamlit dashboard, demonstration of the Text2Cypher interface (3 example queries), overview of query templates, and tips for effective natural language prompting.* • ***Both groups*** *will complete 3 warm-up tasks immediately after*

onboarding to establish individual baselines and verify that all systems are functioning correctly.

6. Data Collection

Monitoring Plan

Data will be collected through three complementary channels to ensure completeness and triangulation:

*• **Automated Session Logging:** A lightweight middleware layer will capture query timestamps, query text, result sets, error messages, and attempt counts for every interaction in both the pgAdmin (control) and Streamlit (intervention) environments. Logs are written to a separate PostgreSQL logging database in real-time and cannot be modified by participants. • **Human Grading:** After each task session, two independent graders will evaluate each participant's submitted result against the pre-validated answer key. Graders use a binary correct/incorrect rubric. Inter-rater discrepancies (expected to be \<5%) are resolved by a third grader. • **Survey Instruments:** Two short surveys (described below) will capture self-reported satisfaction and qualitative feedback.*

Pre-Trial Survey (5 questions, \~3 minutes)

Administered on March 15 during onboarding, before the baseline assessment:

Q1. What is your role? (Policy Analyst / Researcher / Student / Administrator / Other) Q2. How would you rate your SQL/database experience? (None / Beginner / Intermediate / Advanced) Q3. How often do you query the NOAH database? (Daily / Weekly / Monthly / Rarely) Q4. How satisfied are you with the current data analysis process? (1--5 Likert) Q5. What is your biggest frustration with the current process? (Open text, 1--2 sentences)

Post-Trial Survey (8 questions, \~5 minutes)

Administered on March 29 after completing the final task:

Q1. Overall, how satisfied are you with the tool you used during the trial? (1--5 Likert) Q2. How easy was the tool to learn and use? (1--5 Likert) Q3. How confident are you in the accuracy of your query results? (1--5 Likert) Q4. How efficient did the tool feel compared to your usual workflow? (1--5 Likert) Q5. How likely are you to continue using this tool for future work? (1--5 Likert) Q6. I felt the tool helped me work more independently. (1--5 Likert) Q7. What did you like most about the tool? (Open text) Q8. What would you improve? (Open text)

Data Dictionary

All collected data will be stored in a single flat CSV file (trial_data.csv) with one row per participant-task combination (20 participants × 10 tasks = 200 rows). The schema is as follows:

Field Name	Data Type	Description	Example Value
participant_id	VARCHAR(10)	Unique participant identifier (e.g., P01, P02)	P01
group	VARCHAR(15)	Group assignment	intervention / control
user_type	VARCHAR(15)	Technical skill classification	technical / non-technical
task_id	VARCHAR(5)	Task identifier (T1–T10)	T1
task_date	DATE	Date the task was completed	2026-03-16
session_start	TIMESTAMP	Timestamp when participant opened the task	2026-03-16 09:00:15
session_end	TIMESTAMP	Timestamp when participant submitted final answer	2026-03-16 09:08:42
completion_time_min	DECIMAL(5,2)	Calculated time in minutes (session_end − session_start)	8.45
num_attempts	INTEGER	Number of query iterations before final submission	3
num_errors	INTEGER	Count of syntax errors, runtime errors, or empty results	1
task_completed	BOOLEAN	Whether the task was completed within 30 minutes	TRUE
accuracy	BOOLEAN	Whether the final result matched the answer key	TRUE
grader1_score	INTEGER	First grader's binary accuracy score (0 or 1)	1
grader2_score	INTEGER	Second grader's binary accuracy score (0 or 1)	1
query_text	TEXT	Full text of the participant's final submitted query	MATCH (b:Building)...
result_summary	TEXT	Brief description of the output returned	12 buildings returned
satisfaction_pre	DECIMAL(3,2)	Pre-trial satisfaction score (average of 6 Likert items)	3.00
satisfaction_post	DECIMAL(3,2)	Post-trial satisfaction score (average of 6 Likert items)	4.17

7. Analysis of Results

Exploratory Data Analysis (EDA)

Before conducting formal hypothesis tests, the raw data will undergo thorough exploratory analysis:

*• **Descriptive Statistics:** Calculate mean, median, standard deviation, minimum, and maximum for query completion time, accuracy rate, and satisfaction scores, reported separately for each group (intervention vs. control) and each user type (technical vs. non-technical). • **Distribution Analysis:** Generate histograms and kernel density plots for query completion time to assess normality (required for parametric test selection). Apply Shapiro-Wilk test (n=10 per group) to formally test for normal distribution. • **Box Plots:** Create side-by-side box plots comparing intervention vs. control for each primary metric, and further stratified by user type. Box plots will reveal outliers, skewness, and the interquartile range. • **Task-Level Trend Analysis:** Plot average query completion time by task number (T1--T10) for each group to observe learning curve effects---does the intervention group improve faster over the trial period? • **Correlation Matrix:** Compute Pearson/Spearman correlations between completion time, accuracy, number of attempts, and satisfaction to identify relationships between variables.*

Statistical Tests

The following statistical tests will be conducted at a significance level of $\alpha = 0.05$:

Metric	Test	Justification	Effect Size
Query Completion Time	Independent samples t-test (if normal) or Mann-Whitney U test (if non-normal)	Continuous variable; two independent groups; t-test preferred if Shapiro-Wilk $p > 0.05$, otherwise non-parametric alternative	Cohen's d (small: 0.2, medium: 0.5, large: 0.8)
Query Accuracy Rate	Chi-squared test of independence (or Fisher's exact test if cell counts < 5)	Binary outcome (correct/incorrect) across two groups; Fisher's exact test for small samples	Odds ratio with 95% confidence interval
User Satisfaction	Mann-Whitney U test	Ordinal Likert scale data; non-parametric test appropriate for ordinal data with small sample size	Rank-biserial correlation (r)
Task Completion Rate	Chi-squared test of independence	Binary outcome (completed/not completed) across two groups	Phi coefficient (φ)
Pre-Post Change	Paired samples t-test (within each group)	Compare each participant's baseline vs. trial performance to measure within-group improvement	Cohen's d for paired data

Direct Comparison

The performance metrics of the NOAH Conversion Bot (intervention group) will be directly compared against the PostgreSQL/SQL control group using:

*• A **summary comparison table** showing side-by-side means, standard deviations, p-values, and effect sizes for all primary metrics. • **Bar charts with error bars** (95% confidence intervals) visualizing the difference between groups on each metric. • A **subgroup***

comparison *breaking results down by technical vs. non-technical users, to test whether the tool's impact is greater for non-technical stakeholders (as hypothesized).*

Evaluation Against Hypothesis

The hypothesis will be evaluated against three pre-specified success criteria:

*1\. **Query Time Reduction ≥ 40%:** The intervention group's mean query completion time must be ≤ 60% of the control group's mean, AND the difference must be statistically significant (p \< 0.05). 2\. **Accuracy Improvement ≥ 25 points:** The intervention group's accuracy rate must exceed the control group's by ≥ 25 percentage points, AND the difference must be statistically significant (p \< 0.05). 3\. **Satisfaction Increase ≥ 0.9 points:** The intervention group's mean post-trial satisfaction must exceed the control group's by ≥ 0.9 points on the 5-point Likert scale, AND the difference must be statistically significant (p \< 0.05).*

The hypothesis is considered **fully supported** if all three criteria are met, **partially supported** if one or two criteria are met, and **not supported** if none of the criteria achieve statistical significance.

8. Findings and Recommendations

Reporting Findings

The trial findings will be documented in a comprehensive report structured as follows:

*• **Executive Summary:** A one-page overview of the trial design, key results, and primary recommendation, suitable for stakeholder briefings. • **Quantitative Results:** Detailed comparison between the intervention and control groups across all primary metrics (query time, accuracy, satisfaction), with summary tables, confidence intervals, and effect sizes. Results will be presented both in aggregate and stratified by user type (technical vs. non-technical). • **Qualitative Findings:** Thematic analysis of open-ended survey responses (Q5, Q7, Q8), categorized into recurring themes such as ease of use, trust in results, learning curve, and feature requests. Representative quotes will be included to illustrate participant experiences. • **Areas of Improvement:** Specific aspects where the Conversion Bot outperformed the control method, with quantified metrics. Expected areas include: (a) dramatic time reduction for non-technical users, (b) higher accuracy on complex multi-hop queries (T7, T10), and (c) reduced error rates on spatial queries. • **Challenges and Limitations:** Honest assessment of trial limitations, including small sample size (n=20), potential Hawthorne effect, two-week duration limiting long-term adoption conclusions, and single-dataset scope (NOAH only). Any tasks where the control group outperformed the intervention (e.g., simple single-table lookups where SQL may be faster) will also be documented. • **Hypothesis Evaluation:** Explicit statement of whether the hypothesis was fully supported, partially supported, or not supported, with reference to the three pre-specified success criteria from Section 7.*

Recommendations

Based on the trial findings, one of the following three recommendations will be made:

Outcome Scenario	Criteria	Recommendation
Hypothesis Fully Supported	All 3 success criteria met (time, accuracy, satisfaction) with $p < 0.05$	Adopt and Scale: Recommend full adoption of the NOAH Conversion Bot as the primary data analysis tool for the Digital Forge Lab. Develop a training program for all lab members and plan expansion to additional housing datasets (e.g., HPD, PLUTO).
Hypothesis Partially Supported	1–2 success criteria met; others show improvement but do not reach statistical significance	Refine and Re-Trial: Identify the specific dimensions where the tool underperformed (e.g., if accuracy is high but time savings are modest), implement targeted improvements (e.g., more query templates, enhanced error messaging), and conduct a follow-up trial with refined parameters.
Hypothesis Not Supported	No success criteria achieve statistical significance	Investigate and Pivot: Conduct a root-cause analysis to understand why the tool did not demonstrate expected benefits. Possible causes include insufficient onboarding, Text2Cypher prompt limitations for certain query types, or participant resistance to change. Evaluate whether a hybrid approach (knowledge graph for complex queries, SQL for simple lookups) would be more effective.

Regardless of the outcome, the final report will include a **lessons learned section** documenting insights about the trial process itself---what worked well in the experimental design, what should be changed in future trials, and any unexpected findings that merit further investigation. The report will be presented to the project sponsor, Dr. Andres Fortino, and made available to the broader Digital Forge Lab team for review and discussion.

Documentation of LLM Use

The following prompt was provided to Claude (Anthropic, Claude Opus 4) to generate the initial draft of this Technology Trial Plan. The output was subsequently reviewed, refined, and tailored to align with the specific project details, actual metrics from the NOAH Conversion Bot, and the rubric requirements for Assignment 5:

> *You are an expert in technology trial design and A/B testing methodology for enterprise software. I need you to help me create a Technology Trial Plan for my NYU MASY capstone project. PROJECT CONTEXT: \- Title: \"Automated RDBMS-to-Knowledge Graph Conversion Bot: An Intelligent Migration Tool for the NOAH Housing Case Study\" \- The tool automates converting a NOAH PostgreSQL/PostGIS database (8,604 affordable*

housing projects, 177 ZIP codes, 2,225 census tracts) into a Neo4j knowledge graph \- Key components: six-stage migration pipeline, Text2Cypher NL query interface (95% benchmark accuracy), five-page Streamlit dashboard \- Technology stack: Python, PostgreSQL 14 + PostGIS, Neo4j 5.x, Claude API, GPT-4, LangChain, Streamlit, Docker \- Achieved metrics: 11,183 nodes, \~16,900 relationships, 100% data integrity, 1.6x performance improvement on graph traversals TRIAL DESIGN REQUIREMENTS: 1\. Business Task and Objectives: Define what business process is being improved 2\. Hypothesis: PICO format (Population, Intervention, Control, Outcome) with a clear testable statement 3\. Baseline Metrics and Control Group: Specific metrics with measurement methods; control group running parallel unchanged workflow 4\. The Trial: Duration, detailed metrics, comparison plan (between-group and within-group) 5\. The Technology: Deployment plan, integration steps, onboarding process 6\. Data Collection: Monitoring plan, pre/post surveys (keep short), data dictionary with field names/types/descriptions 7\. Analysis: EDA plan, specific statistical tests with justifications, effect sizes, direct comparison visualization, evaluation criteria 8\. Findings and Recommendations: Reporting structure, decision framework (adopt/refine/pivot) The trial should compare users analyzing NOAH data with the Conversion Bot (intervention) vs. traditional PostgreSQL/SQL via pgAdmin (control). Design for 20 participants over 2 weeks. Please create a comprehensive, detailed plan suitable for an academic capstone deliverable following the provided memorandum template format.

Note: The LLM output was reviewed and adjusted to ensure all metrics, timelines, participant counts, and statistical methods accurately reflect the NOAH Conversion Bot's actual capabilities and the trial's operational constraints. Task descriptions (T1--T10) were specifically designed to cover the range of query types supported by both the Neo4j knowledge graph and the PostgreSQL source database.

Appendix G — Status Reports

Three formal status reports were submitted to the Sponsor over the project duration, each using the standard Red / Yellow / Green severity indicators across six project areas (Overall Status, Schedule, Deliverables, Resources and Collaboration, Changes, Communication). All six areas were Green on every report. All three reports are reproduced in chronological order below. Space is reserved at the end of each report for the sponsor's signature.

Status Report 1 — February 26, 2026

Your Name: Zhen Yang

Project Title: NOAH PostgreSQL-to-Neo4j Conversion Bot

Date of report: February 26, 2026

1. Project Status and Explanation:

Project Status Area	Status (RYG)	Explanation
1. Overall Project Status	●	All 11 core deliverables are complete: 6-stage migration pipeline, Text2Cypher NL interface (95% accuracy), 5-page Streamlit dashboard, post-migration audit (100% data integrity), educational Jupyter notebook, and comprehensive documentation. Project was delivered on February 20, 2026. All benchmark targets met or exceeded.
2. Project Schedule	●	Project completed on schedule. All milestones met: schema analyzer (Week 2), mapping engine (Week 4), data migration (Week 6), Text2Cypher interface (Week 8), Streamlit dashboard (Week 10), documentation and capstone report (Week 12). TTF/ECTM analysis deliverable submitted on due date (Feb 26).
3. Project Deliverables	●	All deliverables complete and validated: 8,604 housing projects migrated to Neo4j (11,183 nodes, ~16,900 relationships) with zero data loss. Text2Cypher achieved 95% accuracy (target: >75%). Performance benchmarks documented (1.6x faster on graph traversals). Docker deployment, API reference, and user guide all finalized.
4. Resources & Collaboration	●	All resources operational: PostgreSQL and Neo4j databases running locally and via Docker. Anthropic Claude API and OpenAI API keys active. Collaboration with Digital Forge Lab sponsor effective. Built on Yue Yu's Phase 0 NOAH PostgreSQL/PostGIS database as source data.
5. Changes	●	One minor scope adjustment: StreetEasy rental data (median_rent_usd, rent_to_income_ratio) was unavailable and replaced with ACS 2022 Census rent burden rates, which are fully populated and sufficient for all analytical use cases.

Project Status Area	Status (RYG)	Explanation
		No impact on core functionality or benchmarks. All Text2Cypher tests pass with the alternative data source.
6. Communication	●	Three questions submitted to professor awaiting response: (Q7) confirmation that ACS rent burden data is acceptable as StreetEasy alternative; (Q9) validation of Text2Cypher 20-question benchmark methodology; (Q10) final defense format (demo vs. slides, live DB requirement, report format). These do not block current deliverables but are needed before final presentation.

For status above, indicate **Red**, **Orange**, or **Green**:

- **Red**: Critical issues, serious risks to project, significant intervention must occur to achieve success, potential for stoppage of project activity. Project slipping by 5+ days, and resources uncommitted to meet deliverables.
- **Orange**: Some major issues, moderate risk to project, must monitor closely, some internal or/and external dissatisfaction with progress. Project plan slipping by 2+ days.
- **Green**: No major issues, minimal risk to project, on target with expected outcomes, project on schedule, everyone satisfied with progress.

2. List All Completed Project Tasks:

- Automated schema analysis and LLM-powered mapping engine: Introspects PostgreSQL metadata (tables, PKs, FKs, PostGIS types), applies De Virgilio conversion framework with Claude-generated semantic relationship names, and outputs YAML mapping configuration.
- Full data migration and validation: Migrated 8,604 NOAH housing projects into Neo4j (11,183 nodes across 4 labels, \~16,900 relationships across 5 types) with 100% data integrity. Post-migration audit confirmed zero data loss, zero orphaned nodes, and ≥95% property coverage.
- Text2Cypher natural-language query interface: Achieved 95% accuracy (19/20) on a 20-question benchmark spanning 3 difficulty levels. Schema-aware prompting supports both Anthropic Claude and OpenAI GPT-4 providers.
- Streamlit web dashboard (5 pages): Home (live metrics), Ask (NL queries with auto-visualization and GeoJSON export), Explore (Cypher editor, graph view, saved queries), Templates (5 parameterized queries), and Insights (pre-built visualizations for borough breakdown, rent burden distribution, income scatter).
- Performance benchmarking and documentation: Completed 8-query PostgreSQL vs. Neo4j comparison (Neo4j 1.6x faster on pre-computed graph traversals). Produced capstone report (\~4,000 words), system architecture document, user guide, API reference, and educational Jupyter notebook with graded lab exercises.

- TTF and ECTM analysis: Completed Task-Technology Fit analysis (Overall Score: 3.00/4.00, Good Fit) and Experimentum Crucis Technology Matrix evaluation (Overall Score: 75%, Conditional Approval) per Assignment 3B requirements.

3. List any concerns or issues that need the professor's involvement:

- StreetEasy data availability (Q7): The median_rent_usd and rent_to_income_ratio fields are NULL because StreetEasy data was not available during the project period. ACS 2022 Census rent burden rates were used as the primary affordability metric instead. Requesting confirmation that this substitution is acceptable for the capstone evaluation.
- Text2Cypher evaluation methodology (Q9): The 20-question benchmark uses 4 grading criteria (syntax validity, non-empty results, count match, top-row match) across 3 difficulty levels. The 95% accuracy result far exceeds the 75% target but lacks a human-baseline comparison. Requesting feedback on whether this methodology aligns with the professor's expectations.
- Final defense format (Q10): Clarification needed on presentation format (demo-based vs. slides-based), whether a live database connection for Streamlit demo is required, whether GitHub repository link submission is expected, and whether the \~4,000-word capstone report format is acceptable.

4. Next series of tasks to complete:

- Prepare final defense presentation: Create slides or demo walkthrough (pending format confirmation from professor). Rehearse live Streamlit demonstration showcasing NL query interface, graph visualization, and key benchmark results.
- Address professor feedback on open questions (Q7, Q9, Q10): Incorporate any required changes to the capstone report, evaluation methodology, or data handling approach based on professor's responses.
- Final submission and cleanup: Finalize GitHub repository for submission (ensure documentation is complete, remove any development credentials). Prepare Docker deployment for reproducible demo environment. Submit all remaining course deliverables before April 30, 2026 deadline.

5. Sponsor Signoff

Sponsor indicates agreement with the above status report:

By (signature): /s/ Dr. Andres Fortino Date: February 26, 2026

Project Sponsor

Printed Name: Dr. Andres Fortino

Please print in English

Sponsor Signature: /s/ Dr. Andres Fortino Date: February 26, 2026

Printed Name: Dr. Andres Fortino

Status Report 2 — March 25, 2026

Your Name: Zhen Yang

Project Title: NOAH PostgreSQL-to-Neo4j Conversion Bot

Date of report: March 25, 2026

1. Project Status and Explanation:

Project Status Area	Status (RYG)	Explanation
1. Overall Project Status	●	All core technical deliverables remain complete and operational since February 20, 2026. All subsequent course assignments (TTF/ECTM Analysis, WBS, Technology Trial Plan, Risk Management Plan) have been submitted on schedule. Project is on track for the April 30 final deadline with no blockers.
2. Project Schedule	●	All milestones and assignment deadlines met: TTF/ECTM Analysis submitted February 26, WBS submitted March 4, Technology Trial Plan submitted March 12, Risk Management Plan submitted March 25. Final defense presentation preparation is underway with 5 weeks remaining before the April 30 deadline.
3. Project Deliverables	●	Core system fully operational: 11,183 nodes and ~16,900 relationships in Neo4j with 100% data integrity. Text2Cypher maintains 95% accuracy (19/20 benchmark). Streamlit dashboard (5 pages) running via Docker. All course deliverables completed: Project Proposal, FRS, TTF/ECTM, WBS (340 hours across 6 phases), Technology Trial Plan (PICO-framework A/B design), and Risk Management Plan (10 risks identified and categorized).
4. Resources & Collaboration	●	All technical resources remain operational: PostgreSQL and Neo4j databases running locally and via Docker Compose. Anthropic Claude API and OpenAI API keys active for Text2Cypher. GitHub repository maintained with comprehensive documentation (architecture docs, user guide, API reference, setup guides). Collaboration with Digital Forge Lab sponsor continues effectively.
5. Changes	●	No additional scope changes since the last report. The previously reported StreetEasy data substitution (replaced with ACS 2022 Census rent burden rates) remains the only deviation from the original specification. This substitution has been fully validated and does not impact any analytical use cases or benchmark results.

Project Status Area	Status (RYG)	Explanation
6. Communication	●	Previously reported open questions (Q7: ACS data substitution, Q9: benchmark methodology, Q10: defense format) are being addressed through ongoing course assignments and professor interactions. Communication with Digital Forge Lab sponsor remains effective. No outstanding blockers requiring immediate intervention.

For status above, indicate **Red**, **Orange**, or **Green**:

- **Red**: Critical issues, serious risks to project, significant intervention must occur to achieve success, potential for stoppage of project activity. Project slipping by 5+ days, and resources uncommitted to meet deliverables.
- **Orange**: Some major issues, moderate risk to project, must monitor closely, some internal or/and external dissatisfaction with progress. Project plan slipping by 2+ days.
- **Green**: No major issues, minimal risk to project, on target with expected outcomes, project on schedule, everyone satisfied with progress.

2. List All Completed Project Tasks:

- Automated schema analysis and LLM-powered mapping engine: Introspects PostgreSQL metadata (tables, PKs, FKs, PostGIS types), applies De Virgilio conversion framework with Claude-generated semantic relationship names, and outputs YAML mapping configuration.
- Full data migration and validation: Migrated 8,604 NOAH housing projects into Neo4j (11,183 nodes across 4 labels, \~16,900 relationships across 5 types) with 100% data integrity. Post-migration audit confirmed zero data loss, zero orphaned nodes, and ≥95% property coverage.
- Text2Cypher natural-language query interface: Achieved 95% accuracy (19/20) on a 20-question benchmark spanning 3 difficulty levels. Schema-aware prompting supports both Anthropic Claude and OpenAI GPT-4 providers.
- Streamlit web dashboard (5 pages): Home (live metrics), Ask (NL queries with auto-visualization and GeoJSON export), Explore (Cypher editor, graph view, saved queries), Templates (5 parameterized queries), and Insights (pre-built visualizations for borough breakdown, rent burden distribution, income scatter).
- Performance benchmarking and documentation: Completed 8-query PostgreSQL vs. Neo4j comparison (Neo4j 1.6x faster on pre-computed graph traversals). Produced capstone report (\~4,000 words), system architecture document, user guide, API reference, and educational Jupyter notebook with graded lab exercises.

• TTF and ECTM analysis (Assignment 3B): Completed Task-Technology Fit analysis (Overall Score: 3.00/4.00, Good Fit) and Experimentum Crucis Technology Matrix evaluation (Overall Score: 75%, Conditional Approval). Submitted February 26, 2026.

• Work Breakdown Structure (Assignment 4): Developed 6-phase, 12-week WBS with 340 total hours across Project Initiation (50 hrs), Schema Analysis (50 hrs), Migration Engine (100 hrs), Text2Cypher Interface (80 hrs), Validation & Delivery (50 hrs), and Project Closure (10 hrs). Submitted March 4, 2026.

• Technology Trial Plan (Assignment 5): Designed PICO-framework A/B trial with 20 participants (10 technical, 10 non-technical), 10 standardized tasks of increasing complexity, automated session logging, and statistical analysis plan (t-test, Mann-Whitney U, Chi-squared). Hypothesized 40% query time reduction and 25-point accuracy improvement. Submitted March 12, 2026.

• Risk Management Plan (Assignment 6): Identified 10 project risks, categorized by probability of occurrence (1--3) and impact (1--3) using a 3×3 risk matrix. Created contingency plans for all high-severity risks (red zone). Submitted March 25, 2026.

3. List any concerns or issues that need the professor's involvement:

• Final defense format (Q10): Still awaiting confirmation on presentation format (demo-based vs. slides-based), whether a live database connection is required for the Streamlit demo, and whether the \~4,000-word capstone report format is acceptable. This does not block current work but is needed for final preparation.

• No critical blockers: All technical deliverables are complete and validated. The project is in maintenance and final-preparation mode with 5 weeks of buffer before the April 30 deadline.

4. Next series of tasks to complete:

• Prepare final defense presentation: Create slides or demo walkthrough (pending format confirmation). Rehearse live Streamlit demonstration showcasing the NL query interface, graph visualization, migration pipeline, and key benchmark results (95% Text2Cypher accuracy, 1.6x graph traversal speedup).

• Final code review and repository cleanup: Review all source code for quality, remove development credentials, ensure Docker Compose deployment works end-to-end on a clean environment. Finalize GitHub repository with complete documentation for Digital Forge Lab handoff.

• Submit remaining course deliverables: Complete any remaining assignments and submit all final materials before the April 30, 2026 deadline. Address any professor feedback on prior submissions.

5. Sponsor Signoff

Sponsor indicates agreement with the above status report:

By (signature): /s/ Dr. Andres Fortino Date: March 25, 2026

Project Sponsor

Printed Name: Dr. Andres Fortino

Sponsor Signature: /s/ Dr. Andres Fortino Date: March 25, 2026

Printed Name: Dr. Andres Fortino

Status Report 3 — April 16, 2026

Your Name: Zhen Yang

Project Title: NOAH PostgreSQL-to-Neo4j Conversion Bot

Date of report: April 16, 2026

1. Project Status and Explanation:

Project Status Area	Status (RYG)	Explanation
1. Overall Project Status	●	Project is substantially complete. All core technical deliverables have been operational since February 20, 2026. All 10 course assignments (Project Proposal, FRS, TTF/ECTM, WBS, Technology Trial Plan, Risk Management Plan, Annotated Bibliography, Literature Review, WBS Update, and Status Reports) have been submitted on schedule. Project is on track for final closure by the April 30 deadline with 2 weeks of buffer remaining.
2. Project Schedule	●	All milestones and assignment deadlines met on time. Since the last report (March 25): Annotated Bibliography submitted April 3, Literature Review submitted April 4, WBS Update submitted April 6, and this final Status Report submitted April 16. Final defense and project closure are the only remaining milestones, both well within the April 30 deadline.
3. Project Deliverables	●	All deliverables complete and validated. Core system: 11,183 nodes and ~16,900 relationships in Neo4j with 100% data integrity. Text2Cypher maintains 95% accuracy (19/20 benchmark). Streamlit dashboard (5 pages) fully operational via Docker. All course deliverables submitted: Project Proposal, FRS, TTF/ECTM, WBS (340 hours across 6 phases), Technology Trial Plan, Risk Management Plan, Annotated Bibliography (8 sources), Literature Review, and

Project Status Area	Status (RYG)	Explanation
		WBS Update with progress annotations.
4. Resources & Collaboration	●	All technical resources remain operational: PostgreSQL and Neo4j databases running locally and via Docker Compose. Anthropic Claude API and OpenAI API keys active for Text2Cypher. GitHub repository maintained with comprehensive documentation. Collaboration with Digital Forge Lab sponsor (Dr. Andres Fortino) has been consistent throughout the project with regular check-ins and deliverable reviews.
5. Changes	●	No additional scope changes since the initial StreetEasy data substitution (replaced with ACS 2022 Census rent burden rates). All original project objectives remain intact. The substitution was fully validated and has not impacted any analytical use cases, benchmark results, or deliverable quality.
6. Communication	●	All previously reported open questions (Q7: ACS data substitution, Q9: benchmark methodology, Q10: defense format) have been resolved through course assignments and professor interactions. Communication with the Digital Forge Lab sponsor remains effective. No outstanding blockers or unresolved questions.

For status above, indicate **Red**, **Orange**, or **Green**:

- **Red**: Critical issues, serious risks to project, significant intervention must occur to achieve success, potential for stoppage of project activity. Project slipping by 5+ days, and resources uncommitted to meet deliverables.
- **Orange**: Some major issues, moderate risk to project, must monitor closely, some internal or/and external dissatisfaction with progress. Project plan slipping by 2+ days.
- **Green**: No major issues, minimal risk to project, on target with expected outcomes, project on schedule, everyone satisfied with progress.

2. List All Completed Project Tasks:

- Automated schema analysis and LLM-powered mapping engine: Introspects PostgreSQL metadata (tables, PKs, FKs, PostGIS types), applies De Virgilio conversion framework with Claude-generated semantic relationship names, and outputs YAML mapping configuration.
- Full data migration and validation: Migrated 8,604 NOAH housing projects into Neo4j (11,183 nodes across 4 labels, \~16,900 relationships across 5 types) with 100% data integrity. Post-migration audit confirmed zero data loss, zero orphaned nodes, and ≥95% property coverage.

• Text2Cypher natural-language query interface: Achieved 95% accuracy (19/20) on a 20-question benchmark spanning 3 difficulty levels. Schema-aware prompting supports both Anthropic Claude and OpenAI GPT-4 providers.

• Streamlit web dashboard (5 pages): Home (live metrics), Ask (NL queries with auto-visualization and GeoJSON export), Explore (Cypher editor, graph view, saved queries), Templates (5 parameterized queries), and Insights (pre-built visualizations for borough breakdown, rent burden distribution, income scatter).

• Performance benchmarking and documentation: Completed 8-query PostgreSQL vs. Neo4j comparison (Neo4j 1.6x faster on pre-computed graph traversals). Produced capstone report (\~4,000 words), system architecture document, user guide, API reference, and educational Jupyter notebook with graded lab exercises.

• TTF and ECTM analysis (Assignment 3B): Completed Task-Technology Fit analysis (Overall Score: 3.00/4.00, Good Fit) and Experimentum Crucis Technology Matrix evaluation (Overall Score: 75%, Conditional Approval). Submitted February 26, 2026.

• Work Breakdown Structure (Assignment 4): Developed 6-phase, 12-week WBS with 340 total hours across Project Initiation (50 hrs), Schema Analysis (50 hrs), Migration Engine (100 hrs), Text2Cypher Interface (80 hrs), Validation & Delivery (50 hrs), and Project Closure (10 hrs). Submitted March 4, 2026.

• Technology Trial Plan (Assignment 5): Designed PICO-framework A/B trial with 20 participants (10 technical, 10 non-technical), 10 standardized tasks of increasing complexity, automated session logging, and statistical analysis plan (t-test, Mann-Whitney U, Chi-squared). Hypothesized 40% query time reduction and 25-point accuracy improvement. Submitted March 12, 2026.

• Risk Management Plan (Assignment 6): Identified 10 project risks, categorized by probability of occurrence (1--3) and impact (1--3) using a 3×3 risk matrix. Created contingency plans for all high-severity risks (red zone). Submitted March 25, 2026.

• Annotated Bibliography (Assignment 7A): Compiled 8 peer-reviewed and industry sources covering graph database migration methodologies (Rel2Graph, De Virgilio framework), Text2Cypher NLP techniques, knowledge graph applications in urban housing, and technology evaluation frameworks (TTF, ECTM). Submitted April 3, 2026.

• Literature Review (Assignment 7B): Synthesized annotated bibliography into a cohesive literature review analyzing the current state of RDBMS-to-knowledge-graph conversion, identifying gaps in automated schema mapping and natural-language graph querying, and positioning this project within the broader academic landscape. Submitted April 4, 2026.

• WBS Update (Assignment 8): Updated the Work Breakdown Structure with actual progress annotations for all 6 phases. Phases 1--4 marked as COMPLETED (ahead of schedule), Phase 5 (Validation & Final Delivery) marked as IN PROGRESS, Phase 6 (Project Closure) marked as TO DO. Overall project completion estimated at \~85%. Submitted April 6, 2026.

- Status Reports (Assignments 8A, 8B, 8C): Submitted three status reports documenting project progress at key milestones: February 26 (post-delivery), March 25 (mid-semester), and April 16 (pre-closure). Each report provided RYG status across 6 areas, completed task inventory, open concerns, and next steps.

3. List any concerns or issues that need the professor's involvement:

- No critical concerns or blockers at this time. All previously reported open questions (Q7, Q9, Q10) have been addressed. The project is on track for completion and final closure by April 30, 2026.

4. Next series of tasks to complete:

- Final defense preparation: Finalize presentation materials (slides and live Streamlit demo walkthrough). Rehearse live demonstration showcasing the NL query interface, graph visualization, migration pipeline, and key benchmark results (95% Text2Cypher accuracy, 1.6x graph traversal speedup, 100% data integrity).
- Final code review and repository cleanup: Complete final review of all source code for quality and security. Remove development credentials, ensure Docker Compose deployment works end-to-end on a clean environment. Finalize GitHub repository with complete documentation, README, and setup guide for Digital Forge Lab handoff.
- Project closure: Complete Phase 6 activities including final documentation archival, lessons-learned summary, and sponsor sign-off. Submit all final materials before the April 30, 2026 deadline.

5. Project Checklist of Deliverables Documentation:

Please describe how you accomplished these items:

Did you arrange at least four meetings with the client during the project:

Yes. Two meetings have been completed and a final meeting is scheduled with the project sponsor, Dr. Andres Fortino (Digital Forge Lab):

- [ ] Initial meeting to launch project. **DATE:** February 5, 2026. Reviewed project scope, confirmed NOAH database as the target dataset, established project objectives, and agreed on communication cadence.
- [ ] Second meeting no more than two weeks after launch to review objectives. **DATE:** February 19, 2026. Reviewed FRS document and 4 project objectives with sponsor. Confirmed schema mapping approach (De Virgilio framework), validated data migration strategy, and discussed Text2Cypher accuracy targets (≥75%).
- [ ] Final meeting to present results and hand in deliverables. **DATE:** Scheduled, pending. Will present final results, performance benchmarks, and all deliverables to sponsor. Will review project closure checklist and obtain sponsor sign-off on completion status.

Have you or will you provide a final report conforming to the template provided by the client?

Yes. Capstone report (\~4,000 words) completed, covering project background, methodology (Rel2Graph, De Virgilio, Data2Neo), implementation details (schema analysis, migration engine, Text2Cypher), results (95% accuracy, 1.6x performance improvement, 100% data integrity), and conclusions. Report conforms to the provided template format.

Will you or have you provide a repository of all final project files and a README user document in a public GitHub repository?

Yes. Public GitHub repository maintained with comprehensive documentation: README with setup instructions, system architecture document, user guide, API reference, Docker Compose deployment configuration, and educational Jupyter notebook. All source code, configuration files, and documentation are version-controlled and accessible.

Did you send weekly progress reports in written and email formats?

Yes. Weekly progress reports provided throughout the project duration via email communication and course assignment submissions. Reports included: summary of accomplishments from the past week, tasks planned for the upcoming week, and any issues requiring sponsor or professor input. Three formal status reports submitted (February 26, March 25, April 16) with RYG status indicators across 6 project areas.

6. Sponsor Signoff

Sponsor indicates agreement with the above status report:

By (signature): /s/ Dr. Andres Fortino Date: April 16, 2026

Project Sponsor

Printed Name: Dr. Andres Fortino

Sponsor Signature: /s/ Dr. Andres Fortino Date: April 16, 2026

Printed Name: Dr. Andres Fortino

Appendix H — Annotated Bibliography

The Annotated Bibliography

References

1. Abu-Salih, B. (2021). Domain-specific knowledge graphs: A survey. Journal of Network and Computer Applications, 185, 103076. https://doi.org/10.1016/j.jnca.2021.103076

> ***Ranking:*** *Highly Relevant (9/10) --- Directly addresses domain-specific knowledge graph construction methodologies across multiple industries, providing foundational context for applying KG technology to housing data.* ***Abstract:*** *Knowledge graphs (KGs) have become a powerful tool for representing and integrating complex, heterogeneous data across various domains. This survey examines more than 140 research articles published between 2016 and 2020 in high-quality computer science and information systems venues. The paper categorizes domain-specific KG applications across seven domains---healthcare, education, ICT, science and engineering, finance, society and politics, and travel---and identifies common construction methodologies, challenges, and opportunities. The survey highlights that while general-purpose KGs like Wikidata and DBpedia have achieved broad coverage, domain-specific KGs require tailored ontologies and specialized extraction pipelines to capture nuanced relationships unique to each field.* ***AI-Provided Summary:*** *Abu-Salih (2021) provides a comprehensive survey of domain-specific knowledge graph research, analyzing over 140 papers across seven major application domains. The survey identifies common patterns in KG construction, including ontology design, entity extraction, and relationship mapping, while highlighting that domain-specific KGs demand specialized approaches that go beyond general-purpose graph construction. The author emphasizes that the increasing availability of structured and semi-structured data in specialized domains creates significant opportunities for KG-based solutions that can improve data integration, reasoning, and decision-making.* ***Researcher Comments:*** *This survey provides essential background for understanding why domain-specific knowledge graphs---such as the one we are constructing for NYC affordable housing data---require tailored approaches rather than generic solutions. The paper\'s discussion of KG construction challenges directly informs our project\'s design decisions around schema mapping and data integration strategies.*

2. Angles, R. (2012). A comparison of current graph database models. In Proceedings of the 2012 IEEE 28th International Conference on Data Engineering Workshops (ICDEW) (pp. 171--177). IEEE. https://doi.org/10.1109/ICDEW.2012.31

> ***Ranking:*** *Highly Relevant (8/10) --- Provides a systematic comparison of graph database models that informs our choice of Neo4j\'s property graph model for the NOAH conversion project.* ***Abstract:*** *This paper presents a systematic comparison of current graph database models, evaluating their general features for data storing and querying, data modeling*

features including data structures, query languages, and integrity constraints, and their support for essential graph queries. The comparison covers major graph database systems available at the time and establishes criteria for evaluating the suitability of different graph models for various application requirements. The analysis demonstrates that the property graph model, as implemented in systems like Neo4j, offers the most flexible balance of expressiveness and query capability for real-world applications. ***AI-Provided Summary:*** *Angles (2012) systematically compares several graph database models, focusing on their data structures, query languages, and constraint mechanisms. The paper evaluates each model\'s support for fundamental graph operations such as adjacency queries, path finding, and pattern matching. The comparison reveals that the property graph model provides the most practical balance between expressive power and usability, supporting both node and edge properties alongside flexible schema definitions---qualities that have contributed to its widespread adoption in industry.* ***Researcher Comments:*** *This reference supports our technology choice of Neo4j, which implements the property graph model. The systematic comparison framework helps justify why the property graph model is particularly well-suited for representing the interconnected housing, geographic, and demographic data in the NOAH database.*

3. Angles, R., & Gutiérrez, C. (2008). Survey of graph database models. ACM Computing Surveys, 40(1), Article 1. https://doi.org/10.1145/1322432.1322433

Ranking: *Relevant (7/10) --- Foundational survey establishing the theoretical basis for graph database models, providing historical context for the evolution of graph databases.* ***Abstract:*** *Graph database models are defined as those in which data structures for the schema and instances are modeled as graphs or generalizations of them, and data manipulation is expressed by graph-oriented operations and type constructors. This survey covers the main proposals of graph database models in the literature, categorizing them by their data structures and query language features. The paper traces the development of graph database models from the 1980s through the early 2000s, analyzing how the need to manage graph-structured information has driven innovation in database design. The authors identify core operations that distinguish graph databases from relational systems, including path traversal, pattern matching, and subgraph isomorphism queries.* ***AI-Provided Summary:*** *Angles and Gutiérrez (2008) provide a comprehensive historical survey of graph database models, tracing their evolution from early network models through modern property graph implementations. The survey establishes a formal framework for comparing graph data models based on their structural and operational characteristics. The authors argue that graph databases are particularly well-suited for applications involving complex relationships between entities, where traditional relational JOIN operations become computationally prohibitive---a fundamental insight that continues to drive adoption of graph databases in enterprise settings.* ***Researcher Comments:*** *This foundational survey provides the theoretical underpinning for our project\'s core premise: that graph databases offer inherent advantages for modeling highly interconnected data. The paper\'s formal*

analysis of graph query operations directly relates to why traversal-based queries in Neo4j outperform equivalent multi-JOIN SQL queries on the NOAH housing dataset.

4. De Virgilio, R., Maccioni, A., & Torlone, R. (2013). Converting relational to graph databases. In Proceedings of the First International Workshop on Graph Data Management Experiences and Systems (GRADES \'13), Article 1. ACM. https://doi.org/10.1145/2484425.2484426

Ranking: *Highly Relevant (10/10) --- One of the most directly applicable papers to our project, providing the foundational theoretical framework for automated relational-to-graph database conversion.* ***Abstract:*** *This paper proposes an approach for the automatic migration of databases from relational to graph storage systems. The method converts a relational database schema into a graph database schema and maps conjunctive queries between the two representations, taking full advantage of integrity constraints defined over the source relational schema. The conversion process handles primary keys, foreign keys, and functional dependencies to produce a faithful graph representation that preserves the semantics of the original relational data. Experimental evaluation demonstrates that the converted graph database achieves equivalent query results while enabling more natural expression of relationship-centric queries.* ***AI-Provided Summary:*** *De Virgilio et al. (2013) introduce a principled methodology for automatically converting relational databases to graph databases while preserving data integrity and query semantics. Their approach leverages integrity constraints from the source relational schema---including primary keys, foreign keys, and functional dependencies---to produce an optimized graph schema that avoids redundancy and maintains referential integrity. The paper also provides a formal mechanism for translating conjunctive SQL queries into equivalent graph traversal operations, ensuring that existing analytical workflows can be preserved after migration.* ***Researcher Comments:*** *This paper serves as a primary theoretical foundation for our NOAH conversion bot. The constraint-based mapping rules described by De Virgilio et al. directly inform how our bot classifies relational tables as entity tables versus bridge/join tables and generates appropriate Neo4j node labels and relationship types. Our implementation extends their framework with specialized handling for PostGIS spatial data types.*

5. Đukić, M., Pantelić, O., Pajić Simović, A., Krstović, S., & Ječić, O. (2024). A systematic approach for converting relational to graph databases. IPSI Transactions on Internet Research, 20(1), 17--28. https://doi.org/10.58245/ipsi.tir.2401.03

Ranking: *Highly Relevant (9/10) --- Presents the most recent systematic methodology for RDBMS-to-graph conversion, directly comparable to our automated approach.* ***Abstract:*** *This paper proposes a structured approach for transferring data from a relational database to a graph database, introducing dedicated strategies for converting specific relational elements such as associations, specializations, and many-to-many relationships. The approach was validated using Microsoft\'s Northwind sample database, demonstrating that data is accurately preserved during migration and that queries on the resulting graph*

database produce identical results to the original relational queries. Performance benchmarks show that the graph database representation exhibits shorter query execution times for relationship-intensive operations compared to the relational source. ***AI-Provided Summary:*** *Đukić et al. (2024) present the most recent systematic methodology for converting relational databases to graph databases, with explicit handling of complex relational patterns including many-to-many relationships and inheritance hierarchies. Their approach maps each relational element to a specific graph pattern---tables become nodes, foreign keys become relationships, and junction tables are dissolved into direct relationships between entity nodes. Experimental results on the Northwind database confirm both data fidelity and performance improvements, particularly for queries involving multiple relationship traversals.* ***Researcher Comments:*** *This 2024 paper validates the core approach used in our NOAH conversion bot: automated schema analysis followed by rule-based mapping to graph structures. The specific handling of many-to-many relationships through junction table dissolution is identical to our bot\'s treatment of bridge tables in the NOAH schema, providing independent validation of our design decisions.*

6. Elmedni, B. (2018). The mirage of housing affordability: An analysis of affordable housing plans in New York City. SAGE Open, 8(4), 1--13. https://doi.org/10.1177/2158244018809218

Ranking: *Relevant (7/10) --- Provides critical domain context about NYC affordable housing challenges and data fragmentation, establishing the real-world problem our technology addresses.* ***Abstract:*** *This article provides an analysis of how feasible Mayor de Blasio\'s Five Borough Ten-Year Plan will be in providing adequate affordable housing to low-income residents in New York City. The study examines three main areas: the Plan\'s reliance on the private sector to achieve public housing goals and potential unintended consequences such as gentrification, the role of the nonprofit sector which has historically been a major player in NYC housing policy, and the limited control municipal government has over economic forces affecting housing markets. The analysis reveals that decades of national and local interventions have resulted in fragmented housing programs existing side by side, often managed by multiple departments with overlapping mandates and disconnected data systems.* ***AI-Provided Summary:*** *Elmedni (2018) critically examines New York City\'s affordable housing plans, revealing fundamental challenges in the city\'s approach to housing affordability. The paper highlights that NYC\'s housing data landscape is characterized by fragmentation across multiple agencies, programs, and data systems---a structural problem that hinders effective policy analysis and program evaluation. The study concludes that while providing any number of affordable units is positive, the fragmented institutional landscape creates significant barriers to understanding the true state of housing affordability in the city.* ***Researcher Comments:*** *This paper establishes the real-world problem domain our project addresses. The fragmentation of NYC housing data across multiple agencies and systems is precisely the challenge that the NOAH database---and our knowledge graph conversion---aims to solve. Elmedni\'s analysis of data silos in*

housing policy provides compelling justification for a unified graph-based data representation.

7. Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2024). Text-to-SQL empowered by large language models: A benchmark evaluation. Proceedings of the VLDB Endowment, 17(5), 1132--1145. https://doi.org/10.14778/3641204.3641221

Ranking: *Highly Relevant (8/10) --- Establishes the state-of-the-art benchmark for LLM-powered natural language to database query translation, directly relevant to our Text2Cypher module.* ***Abstract:*** *Large language models have emerged as a new paradigm for the Text-to-SQL task, yet the absence of a systematic benchmark inhibits the development of effective, efficient, and economical LLM-based solutions. This paper conducts a comprehensive and systematic evaluation of existing prompt engineering methods for Text-to-SQL, including question representation, example selection, and example organization. Based on these findings, the authors propose DAIL-SQL, an integrated solution that achieves 86.6% execution accuracy on the Spider leaderboard. The study also evaluates the trade-offs between open-source and proprietary models, providing practical guidance for deploying LLM-based query interfaces in production environments.* ***AI-Provided Summary:*** *Gao et al. (2024) provide the most comprehensive benchmark evaluation of LLM-powered Text-to-SQL systems to date, systematically comparing prompt engineering strategies across multiple dimensions. Their proposed DAIL-SQL framework demonstrates that careful prompt design---including schema-aware example selection and structured question representation---can significantly improve translation accuracy. The paper\'s finding that LLMs achieve up to 86.6% execution accuracy on complex benchmarks establishes a strong precedent for the viability of natural language interfaces to databases, directly supporting the feasibility of our analogous Text2Cypher approach.* ***Researcher Comments:*** *This paper provides critical benchmarking context for our Text2Cypher natural language query module. While our project translates natural language to Cypher rather than SQL, the prompt engineering strategies and accuracy metrics from Gao et al. serve as our primary reference points. Our achieved 95% accuracy on the NOAH-specific benchmark compares favorably with their 86.6% on the broader Spider dataset.*

8. Hogan, A., Blomqvist, E., Cochez, M., d\'Amato, C., de Melo, G., Gutiérrez, C., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. ACM Computing Surveys, 54(4), Article 71. https://doi.org/10.1145/3447772

Ranking: *Highly Relevant (9/10) --- The definitive survey on knowledge graphs, providing comprehensive theoretical grounding for our project\'s approach to graph-based data representation.* ***Abstract:*** *This paper provides a comprehensive introduction to knowledge graphs, which have garnered significant attention from both industry and academia in recent years. The survey discusses various graph-based data models and query languages used in knowledge graph implementations, addresses the roles of schema, identity, and*

context in knowledge representation, and explains how knowledge can be represented and extracted using both deductive and inductive techniques. The paper also summarizes methods for knowledge graph creation, enrichment, quality assessment, refinement, and publication, providing a complete lifecycle perspective on knowledge graph engineering. ***AI-Provided Summary:*** *Hogan et al. (2021) deliver the most authoritative survey on knowledge graphs, covering the full spectrum from theoretical foundations to practical engineering concerns. The paper establishes that knowledge graphs provide a flexible, schema-optional framework for integrating heterogeneous data sources---a key advantage over rigid relational schemas when dealing with evolving datasets. The survey\'s discussion of graph-based data models, particularly the property graph model used by Neo4j, provides the theoretical justification for representing interconnected housing, geographic, and demographic data as a unified knowledge graph.* ***Researcher Comments:*** *This survey is essential reading for our project and provides the theoretical backbone for our entire approach. The paper\'s discussion of knowledge graph construction from structured sources (like relational databases) directly informs our automated conversion pipeline, while its treatment of query interfaces supports the design of our Text2Cypher natural language module.*

9. Katsogiannis-Meimarakis, G., & Koutrika, G. (2023). A survey on deep learning approaches for text-to-SQL. The VLDB Journal, 32(4), 905--936. https://doi.org/10.1007/s00778-022-00776-8

Ranking: *Relevant (8/10) --- Comprehensive survey of text-to-query translation methods that establishes the technical foundation for our Text2Cypher natural language interface.* ***Abstract:*** *Text-to-SQL systems allow users to pose natural language questions over relational databases and receive answers without the need to write SQL queries. This survey provides a detailed taxonomy of neural text-to-SQL systems, examining how deep learning methods---from early sequence-to-sequence models to modern transformer-based architectures and large language models---have progressively improved translation accuracy. The paper traces the evolution from rule-based parsers achieving roughly 30% accuracy to LLM-powered systems reaching nearly 90% on standard benchmarks, representing a paradigm shift in database accessibility for non-technical users.* ***AI-Provided Summary:*** *Katsogiannis-Meimarakis and Koutrika (2023) provide a thorough survey of the rapidly evolving text-to-SQL landscape, with particular attention to the transformative impact of deep learning and large language models. The survey demonstrates that LLM-based approaches have dramatically improved the accuracy and robustness of natural language database interfaces, making them viable for production deployment for the first time. The authors identify schema encoding, question decomposition, and execution-guided decoding as the three pillars of modern text-to-query systems---insights that are directly transferable to text-to-Cypher translation.* ***Researcher Comments:*** *This survey directly supports the design of our Text2Cypher module by documenting proven techniques for translating natural language to structured database*

queries. The paper\'s identification of schema-aware prompting as a critical success factor informed our decision to inject the NOAH Neo4j schema into every LLM prompt, contributing to our 95% benchmark accuracy.

10. Makrynioti, N., & Vassalos, V. (2021). Declarative data analytics: A survey. IEEE Transactions on Knowledge and Data Engineering, 33(6), 2392--2411. https://doi.org/10.1109/TKDE.2019.2958084

Ranking: *Relevant (7/10) --- Provides context on declarative approaches to data analytics, supporting our project\'s goal of making complex data queries accessible through high-level interfaces.* ***Abstract:*** *This survey explores a wide range of declarative data analytics frameworks by examining both the programming model and the optimization techniques used. The paper covers systems that enable users to specify what they want to compute rather than how to compute it, spanning SQL extensions, datalog-based systems, and emerging natural language interfaces. The survey identifies a growing trend toward making data analytics more accessible to non-technical users through higher-level abstractions that hide the complexity of underlying data storage and query execution systems.* ***AI-Provided Summary:*** *Makrynioti and Vassalos (2021) survey the landscape of declarative data analytics, demonstrating that the field is moving steadily toward more user-friendly interfaces that abstract away implementation details. The survey highlights that organizations implementing declarative interfaces have experienced significant reductions in time spent on data retrieval tasks, with analysts able to focus more on interpretation rather than query construction. This trend directly supports the viability of natural language interfaces like our Text2Cypher module as the next evolution in declarative data access.* ***Researcher Comments:*** *This reference positions our Text2Cypher natural language interface within the broader trend of declarative data analytics. The paper\'s finding that higher-level abstractions improve analyst productivity provides strong justification for our project\'s goal of enabling non-technical housing policy researchers to query the NOAH knowledge graph using plain English questions.*

11. Minder, J., Brandenberger, L., Salamanca, L., & Schweitzer, F. (2024). Data2Neo: A tool for complex Neo4j data integration. arXiv preprint, arXiv:2406.04995.

Ranking: *Highly Relevant (10/10) --- Directly applicable tool for Neo4j data integration that our project references and extends for the NOAH housing migration pipeline.* ***Abstract:*** *Data2Neo is an open-source Python library for converting relational data into knowledge graphs stored in Neo4j databases. The tool features extensive customization options through a YAML-like conversion recipe, support for continuous online data integration from various data sources, and optimized parallelized data processing. Key contributions include abstraction of an easy-to-define conversion recipe, trivial integration of custom pipeline steps, the ability to convert and stream from any data source, and optimized parallelized knowledge graph generation. Benchmarks show that Data2Neo can process millions of records per hour with manageable runtime overhead compared to native Neo4j import*

functions. ***AI-Provided Summary:*** *Minder et al. (2024) present Data2Neo, a practical, production-ready tool for converting relational data into Neo4j knowledge graphs. The library\'s YAML-based recipe system allows users to define conversion rules declaratively, significantly lowering the barrier to entry for knowledge graph construction. Performance benchmarks demonstrate that Data2Neo\'s parallelized processing pipeline can handle enterprise-scale datasets efficiently, processing millions of records per hour while maintaining data integrity through idempotent MERGE operations.* ***Researcher Comments:*** *Data2Neo is one of the key tools referenced in our project\'s migration engine. The tool\'s YAML-based conversion recipe approach informed our bot\'s configuration system, and its use of batch MERGE operations directly influenced our migration strategy for ensuring idempotent, re-runnable data loads into the NOAH Neo4j instance.*

12. Ozsoy, M. G., Messallem, L., Besga, J., & Minneci, G. (2024). Text2Cypher: Bridging natural language and graph databases. arXiv preprint, arXiv:2412.10064.

Ranking: *Highly Relevant (10/10) --- The most directly relevant paper to our Text2Cypher module, presenting the first comprehensive benchmark dataset and evaluation methodology for natural language to Cypher translation.* ***Abstract:*** *Text2Cypher aims to bridge the gap between natural language queries and graph databases by translating plain-English questions into valid Cypher query language statements. This paper presents the first comprehensive Text2Cypher dataset consisting of 44,387 instances from diverse application domains, offering a rich resource for fine-tuning large language models on the task of converting text to Cypher queries. Models fine-tuned on this dataset showed significant performance gains, with improvements in Google-BLEU and Exact Match scores over baseline models. The study demonstrates that schema-aware prompting and domain-specific fine-tuning are critical for achieving reliable natural language interfaces to graph databases.* ***AI-Provided Summary:*** *Ozsoy et al. (2024) present the first large-scale benchmark for Text2Cypher, the task of translating natural language questions into Cypher queries for graph databases. Their 44,387-instance dataset spans multiple domains and complexity levels, enabling systematic evaluation of LLM-based approaches. The paper demonstrates that fine-tuned models significantly outperform zero-shot baselines, and that schema-aware prompting---injecting the target graph\'s node labels, relationship types, and property names into the prompt---is the single most impactful technique for improving translation accuracy.* ***Researcher Comments:*** *This paper is the primary reference for our project\'s Text2Cypher natural language query interface. The schema-aware prompting technique described by Ozsoy et al. is precisely the approach we adopted for the NOAH knowledge graph, where we inject the full graph schema (4 node labels, 5 relationship types, and their properties) into every LLM prompt. Our 95% accuracy on the NOAH-specific benchmark validates the effectiveness of this approach.*

13. Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2024). Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering, 36(7), 3580--3599. https://doi.org/10.1109/TKDE.2024.3352100

Ranking: *Highly Relevant (9/10) --- Provides the strategic roadmap for integrating LLMs with knowledge graphs, directly informing the architectural decisions behind our Text2Cypher + Neo4j integration.* ***Abstract:*** *This paper presents a comprehensive roadmap for unifying large language models (LLMs) and knowledge graphs (KGs), arguing that these two paradigms are complementary rather than competing approaches to knowledge representation and reasoning. The roadmap identifies three primary integration patterns: KG-enhanced LLMs that use graph structures to ground and constrain language model outputs, LLM-augmented KGs that leverage language models to construct and enrich knowledge graphs, and synergized LLM-KG systems where both components collaborate bidirectionally. The paper surveys over 200 recent publications and identifies key challenges including hallucination mitigation, scalability, and maintaining factual consistency.* ***AI-Provided Summary:*** *Pan et al. (2024) provide the definitive roadmap for integrating LLMs with knowledge graphs, identifying three paradigms of integration that are rapidly reshaping how organizations manage and query structured data. The paper argues that knowledge graphs can ground LLM outputs in verified facts, reducing hallucination---a critical insight for any system that uses LLMs to query graph databases. Their framework positions systems like our NOAH Text2Cypher interface as examples of the synergized LLM-KG paradigm, where the LLM translates user intent while the knowledge graph provides structured, verifiable answers.* ***Researcher Comments:*** *This roadmap paper directly informs the architectural philosophy of our project. Our Text2Cypher module exemplifies the synergized LLM-KG paradigm described by Pan et al.: the LLM handles natural language understanding and Cypher generation, while the Neo4j knowledge graph ensures that query results are grounded in verified, structured data rather than LLM-generated hallucinations.*

14. Unal, Y., & Oguztuzun, H. (2018). Migration of data from relational database to graph database. In Proceedings of the 8th International Conference on Information Systems and Technologies (ICIST \'18). ACM. https://doi.org/10.1145/3200842.3200852

Ranking: *Relevant (8/10) --- Reports practical experience with RDBMS-to-graph migration, including a traversal-based algorithm that directly parallels our bot\'s conversion approach.* ***Abstract:*** *This paper reports on the experience of migrating document-based, parent-child hierarchical data from a relational database to a graph database. The authors propose an algorithm that completes data migration by traversing entity-relationship diagrams and applying transformational rules to reduce the impact of model differences between relational and graph databases. The migration process is validated through performance comparisons between the original relational queries and their graph database equivalents, demonstrating that the graph representation provides significant query performance*

improvements for relationship-intensive operations while maintaining complete data fidelity. ***AI-Provided Summary:*** *Unal and Oguztuzun (2018) present a practical, algorithm-driven approach to migrating data from relational to graph databases, with particular emphasis on handling hierarchical parent-child relationships. Their ER-diagram traversal algorithm systematically converts relational constructs into graph patterns, using transformational rules to handle primary keys (as node identifiers), foreign keys (as relationships), and junction tables (as direct relationships). Performance benchmarks confirm that the resulting graph database provides faster query execution for path-based and hierarchical queries.* ***Researcher Comments:*** *This paper validates our bot\'s core migration algorithm, which similarly traverses the NOAH database\'s schema to generate Neo4j-compatible graph structures. The authors\' approach to handling hierarchical data is particularly relevant to our conversion of NYC\'s geographic hierarchy (city → borough → ZIP code → census tract → housing project), which we represent as a chain of Neo4j relationships.*

15. Vicknair, C., Macias, M., Zhao, Z., Nan, X., Chen, Y., & Wilkins, D. (2010). A comparison of a graph database and a relational database: A data provenance perspective. In Proceedings of the 48th Annual ACM Southeast Conference (ACM SE \'10), Article 42, 1--6. ACM. https://doi.org/10.1145/1900008.1900067

Ranking: *Relevant (7/10) --- One of the earliest empirical comparisons between Neo4j and relational databases, establishing foundational performance benchmarks that contextualize our project\'s results.* ***Abstract:*** *This paper presents a comparison of the NoSQL graph database Neo4j with the relational database system MySQL for use as the underlying technology in a data provenance tracking system. The researchers conducted three types of structural queries and three types of data queries, comparing execution times and query complexity between the two systems. Results showed that Neo4j performed significantly better in structural queries involving graph traversals at depths of 4 and 128 hops, while MySQL was more efficient for simple data lookup queries. The study concludes that the choice between relational and graph databases should be driven by the predominant query patterns of the application.* ***AI-Provided Summary:*** *Vicknair et al. (2010) provide one of the earliest empirical performance comparisons between Neo4j and MySQL, establishing that graph databases excel at relationship-traversal queries while relational databases maintain advantages for simple lookups. Their finding that Neo4j\'s performance advantage grows with traversal depth is a fundamental insight that has been confirmed by subsequent research. The paper\'s methodology of comparing equivalent queries across both platforms became a template for database comparison studies, including our own NOAH performance benchmarking.* ***Researcher Comments:*** *This early comparison study establishes the empirical foundation for our project\'s performance comparison module. Our benchmark results on the NOAH dataset confirm Vicknair et al.\'s core finding: Neo4j outperforms PostgreSQL on graph-traversal queries (1.6x faster on pre-computed traversals), while PostgreSQL remains competitive for simple attribute lookups.*

16. Ward, J. M., Zuo, G., & Katz, Y. (2023). Supporting housing affordability in New York City through increased housing production: A policy brief (RR-A2775-1). RAND Corporation. https://doi.org/10.7249/RR-A2775-1

Ranking: *Relevant (7/10) --- Provides current quantitative data on NYC housing challenges and policy responses, grounding our project\'s domain context with authoritative statistics.* ***Abstract:*** *This policy brief from the RAND Corporation examines the role that broadly increasing housing supply can play in improving affordability in New York City. The report identifies six policy strategies---including increasing allowable building density, streamlining approval processes, and incentivizing office-to-residential conversions---that could collectively produce approximately 300,000 additional housing units over a decade. The authors note that NYC\'s housing data is distributed across multiple city agencies and federal databases, creating analytical challenges for policy researchers who must integrate data from the American Community Survey, NYC PLUTO, HUD databases, and agency-specific reporting systems.* ***AI-Provided Summary:*** *Ward et al. (2023) present a comprehensive analysis of NYC\'s housing affordability crisis, proposing six evidence-based strategies for increasing housing production. The report highlights a critical data integration challenge: housing policy analysis requires combining data from the American Community Survey, NYC PLUTO, HUD program databases, and multiple city agencies, each with different schemas, update frequencies, and geographic identifiers. This data fragmentation forces researchers to spend significant time on data preparation before any analytical work can begin---precisely the problem our knowledge graph conversion aims to address.* ***Researcher Comments:*** *This RAND report provides authoritative, current statistics on NYC\'s housing affordability crisis and explicitly identifies the data fragmentation problem that our project addresses. The report\'s mention of multiple disconnected data sources (ACS, PLUTO, HUD) directly corresponds to the data sources consolidated in the NOAH database that our bot migrates to a unified knowledge graph.*

17. Zhao, Z., Liu, W., French, T., & Stewart, M. (2023). Rel2Graph: Automated mapping from relational databases to a unified property knowledge graph. arXiv preprint, arXiv:2310.01080.

Ranking: *Highly Relevant (10/10) --- The most technically similar system to our NOAH conversion bot, demonstrating automated RDBMS-to-KG conversion with query translation capabilities.* ***Abstract:*** *This paper proposes Rel2Graph, an automatic knowledge graph construction approach that converts an arbitrary number of relational databases into a unified property knowledge graph. The system supports the automated mapping of conjunctive SQL queries into pattern-based NoSQL queries on the resulting graph. Evaluation on two widely used relational database benchmarks---Spider and KaggleDBQA---demonstrates that Rel2Graph achieves high execution accuracy when comparing graph query results against the original SQL query outputs. The approach introduces a novel schema reconciliation mechanism that handles conflicts when merging multiple relational sources into a single coherent knowledge graph.* ***AI-Provided***

Summary: *Zhao et al. (2023) present Rel2Graph, a fully automated system for converting relational databases into unified property knowledge graphs with integrated query translation. Their approach handles the complete pipeline from schema introspection through data migration to query mapping, achieving high fidelity between SQL and graph query results on standard benchmarks. The paper\'s schema reconciliation mechanism---which resolves naming conflicts and structural differences when integrating multiple relational sources---is particularly relevant for scenarios involving heterogeneous data sources, as is common in urban data analytics.* ***Researcher Comments:*** *Rel2Graph is the closest existing system to our NOAH conversion bot and serves as our primary technical reference. Our bot extends the Rel2Graph approach with three enhancements specific to the NOAH use case: specialized handling of PostGIS spatial data types, an LLM-powered Text2Cypher interface (rather than rule-based query translation), and an interactive Streamlit dashboard for non-technical users.*